Productionising Models

Introduction

Productionising Models

Overview

Agenda

- [08:15 – 09:00] – Lecture: DORA metrics and productionising models
- [09:15 – 12:00] – Exercises: MLflow catch-up, custom model wrapper, model serving and monitoring

Preparation

For the exercises:

- https://MLflow.org/docs/latest/ml/ (use as reference material, don't do exercises)
- https://fastapi.tiangolo.com/ (use as reference material, don't do the tutorial)

For the lecture:

- https://www.mlebook.com/wiki/doku.php (Chapter 9)
- https://ml-ops.org/content/mlops-principles (Monitoring in particular)
- https://www.datadoghq.com/knowledge-center/dora-metrics/ (or other DORA metric google searches is fine)
- https://neptune.ai/blog/how-to-monitor-your-models-in-production-guide (skimming it is fine)
- https://dvc.org/doc/use-cases/experiment-tracking (skimming it is fine)

- Overview of different MLOps tools (skimming it is fine)

Notes

- The exercises start with the Exercise Intro page: make sure MLflow works for you (see Exercise 0 there).

Lecture Intro

Productionising Models

Data Science in Production: MLOps and Software Engineering

Outline

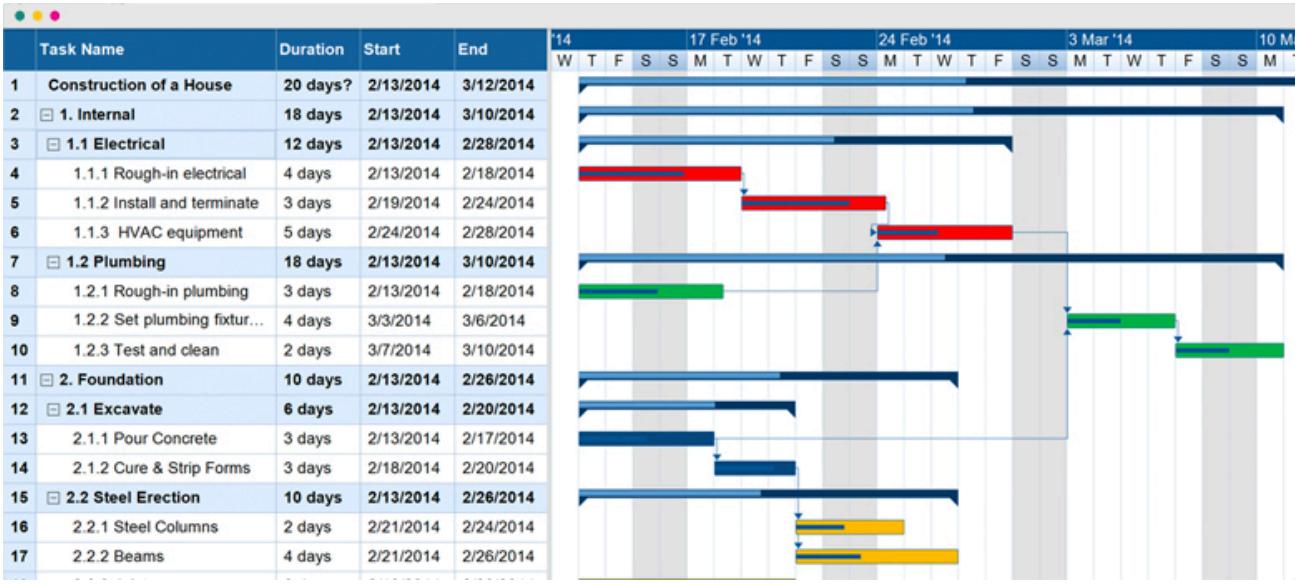
1. DORA metrics (for DevOps)
2. DORA metrics (for MLOps)
3. Deployment frequency
4. Lead time for change
5. Mean time to restore
6. Change failure rate





Developing a(ny) software product

Setting the context



	Task Name	Duration	Start	End
1	Construction of a House	20 days?	2/13/2014	3/12/2014
2	1. Internal	18 days	2/13/2014	3/10/2014
3	1.1 Electrical	12 days	2/13/2014	2/28/2014
4	1.1.1 Rough-in electrical	4 days	2/13/2014	2/18/2014
5	1.1.2 Install and terminate	3 days	2/19/2014	2/24/2014
6	1.1.3 HVAC equipment	5 days	2/24/2014	2/28/2014
7	1.2 Plumbing	18 days	2/13/2014	3/10/2014
8	1.2.1 Rough-in plumbing	3 days	2/13/2014	2/18/2014
9	1.2.2 Set plumbing fixtur...	4 days	3/3/2014	3/6/2014
10	1.2.3 Test and clean	2 days	3/7/2014	3/10/2014
11	2. Foundation	10 days	2/13/2014	2/26/2014
12	2.1 Excavate	6 days	2/13/2014	2/20/2014
13	2.1.1 Pour Concrete	3 days	2/13/2014	2/17/2014
14	2.1.2 Cure & Strip Forms	3 days	2/18/2014	2/20/2014
15	2.2 Steel Erection	10 days	2/13/2014	2/26/2014
16	2.2.1 Steel Columns	2 days	2/21/2014	2/24/2014
17	2.2.2 Beams	4 days	2/21/2014	2/26/2014

Whenever you are working on a project, you want to make sure that you deliver the right thing on time. Unsurprisingly, there are many different tools and methods available to ensure the success of the project and team.

An example of a project management tool is a Gantt chart (as shown above), which many of you may have seen in some form or another. It can be useful for projects where the completion of one step is a prerequisite for the next steps.

Software team performance

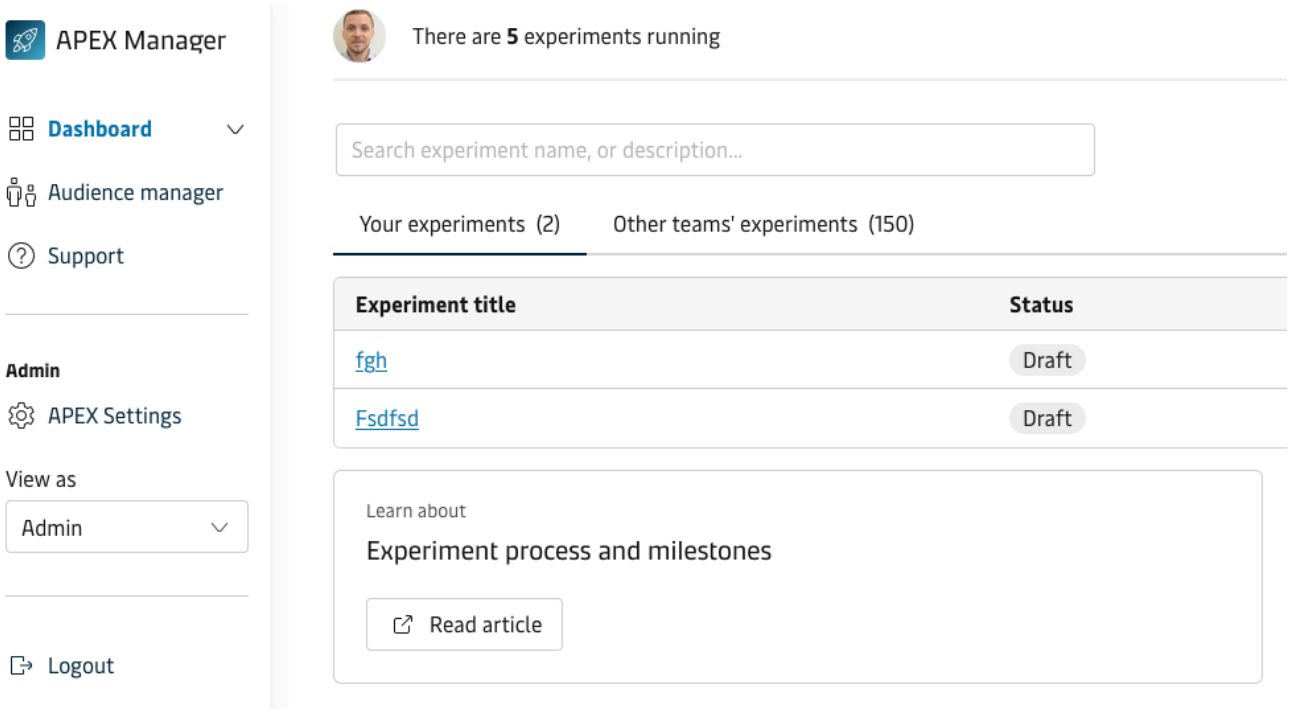
Software projects, however, tend to be less linear, less waterfall'y, and have different risks than for example construction of a house. And AI/ML projects? A whole other set of challenges with data and models.

So how can we know if the project and team are performing as expected?

While we will talk more about working in a team and agile processes later, one tool for evaluating the performance of the team is *DORA metrics*.

In the following sections we will cover what they mean for any software team before relating it to AI/ML projects.

Example: Apex Manager



An example from my work is the **APEX team**: a team maintaining an internal A/B testing tool.

They continuously deliver new requests, such as:

- **Features**: new functionality in the tool, like the Audience Manager, improved statistical methods, or new results UI.
- **Bugs**: fixes to either the experiment setup (involving other teams) or in own product (all software has bugs).

- **Resuming work**: re-opening experiments that were parked

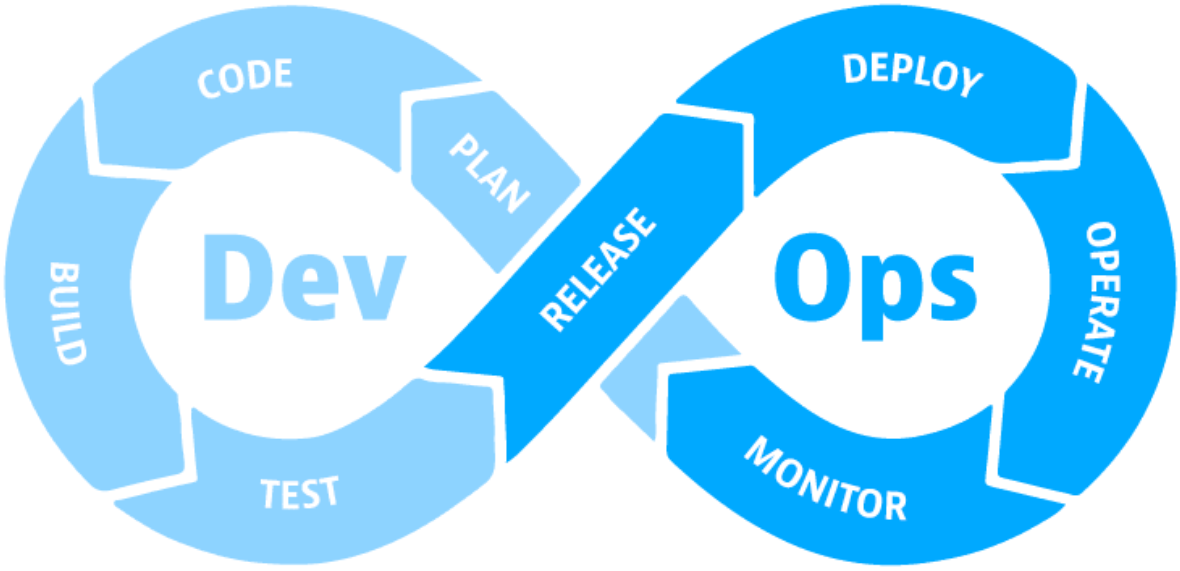
Multiple people contribute changes to the same tool, at different speeds, with different levels of context. That is exactly the situation the DORA metrics are designed to shed light on: how fast does work get to users, and how safely does it land there?

DORA Metrics

DORA Metrics

DORA is the Google team **D**ev**O**ps **R**esearch and **A**ssessment, who publish the yearly "State of DevOps" reports (since ~2013).

> ***Note*** *DORA is not an abbreviation of the metrics themselves – it is the name of the research team behind them.*



Do you have any ideas what they could be?

Overview

Four metrics for measuring success/performance of software delivery:

- 2 "dev" metrics
- 2 "ops" metrics

Metric	DevOps
Deployment Frequency	How often does your organization deploy code to production or release it to end-users?
Lead Time for Changes	How long does it take to go from code committed to code successfully running in production?
Mean Time To Restore (MTTR)	How long does it generally take to restore service when a service incident or a defect that impacts users occurs (e.g., unplanned outage or service impairment)?
Change Failure Rate	What percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?

From https://ml-ops.org/content/mlops-principles

Deployment frequency

How often you deploy code to end-users.

- Smaller bits > large chunks
- Typically ~1/week up to daily

Lead time for changes

The time from commit to production.

Highlights inefficient processes:

- Slow code review?
- (Too) complex code?

From 1 week/month down to <1 day.

Mean time to restore

How quickly you recover when something breaks: fix bugs → new code → update.

You don't want bugs in production…

From months down to <1 hour.

Change failure rate

How much of the deployed code is "faulty" production code.

- Nothing is perfect
- But a high rate → low trust

From ~0% up to +60%.

Shortcomings

Goodhart's law

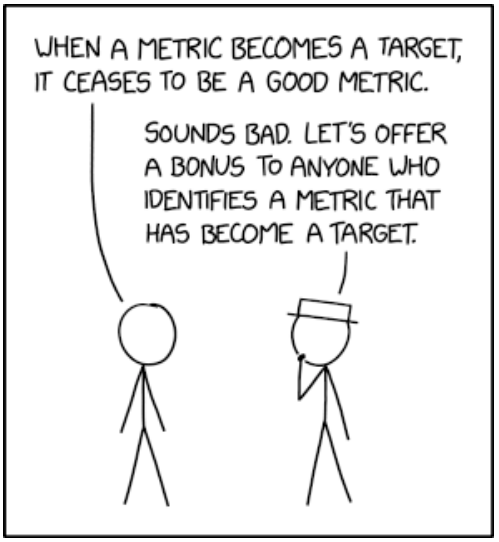
"*When a measure becomes a target, it ceases to be a good measure.*" Optimising for the metric is not the same as optimising for good software delivery.

Don't compare teams

The metrics are context specific – comparing teams against each other on DORA metrics is a misuse.

Simplification

Four metrics is a simplification – DORA themselves track 24 other metrics in their reports.



> *"I'm pleased to report we're now identifying and replacing hundreds of outdated metrics per hour."*

From https://xkcd.com/2899/

DORA for MLOps

DORA... for MLOps?

MLOps?

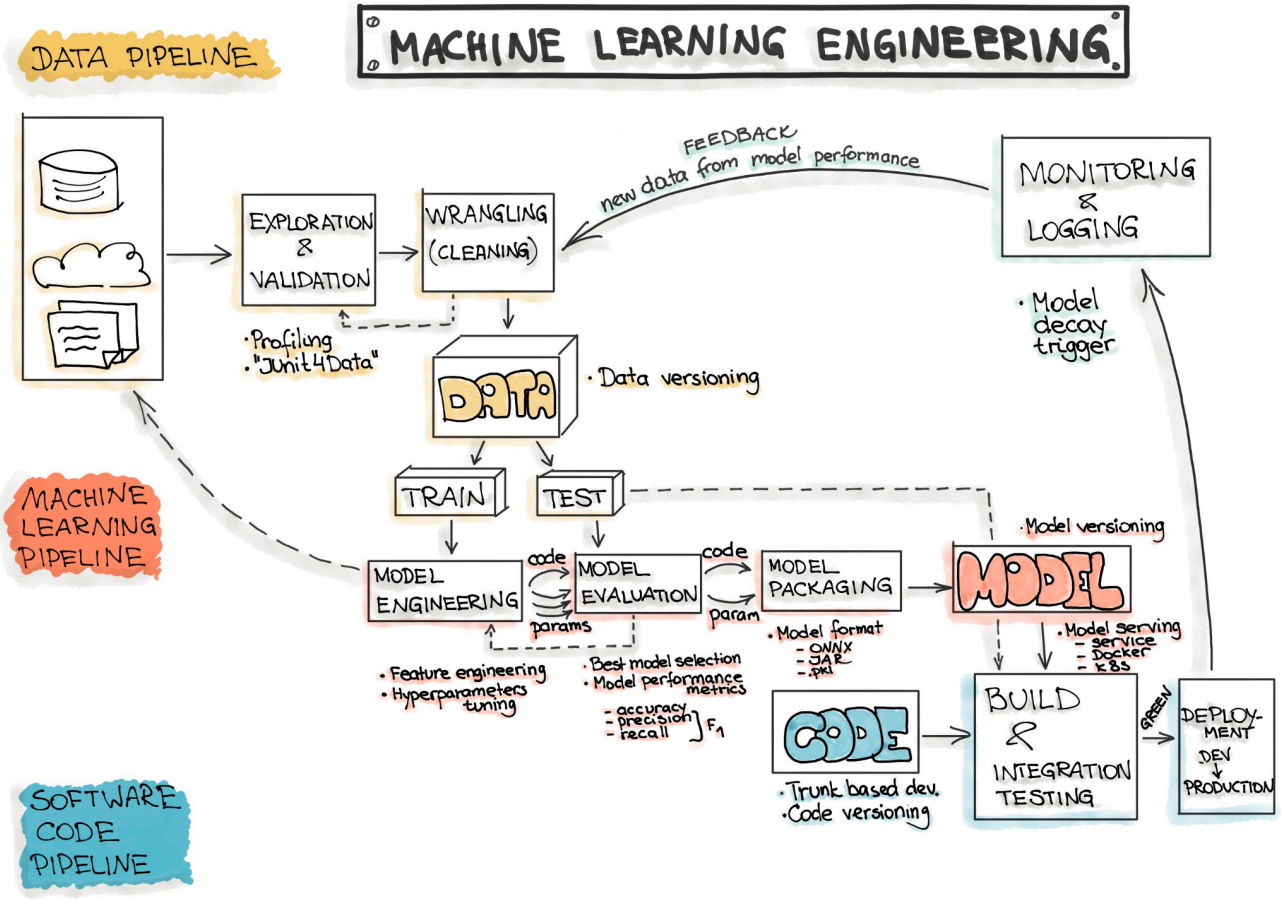
The four DORA metrics were made for DevOps – software delivery in general. But we are running **machine learning** systems, which have more moving parts than a regular software project.

Metric	DevOps	MLOps
Deployment Frequency	How often does your organization deploy code to production or release it to end-users?	ML Model Deployment Frequency depends on 1) Model retraining requirements (ranging from less frequent to online training). Two aspects are crucial for model retraining 1.1) Model decay metric. 1.2) New data availability. 2) The level of automation of the deployment process, which might range between *manual deployment* and *fully automated CI/CD pipeline*.
Lead Time for Changes	How long does it take to go from code committed to code successfully running in production?	ML Model Lead Time for Changes depends on 1) Duration of the explorative phase in Data Science in order to finalize the ML model for deployment/serving. 2) Duration of the ML model training. 3) The number and duration of manual steps during the deployment process.
Mean Time To Restore (MTTR)	How long does it generally take to restore service when a service incident or a defect that impacts users occurs (e.g., unplanned outage or service impairment)?	ML Model MTTR depends on the number and duration of manually performed model debugging, and model deployment steps. In case, when the ML model should be retrained, then MTTR also depends on the duration of the ML model training. Alternatively, MTTR refers to the duration of the rollback of the ML model to the previous version.
Change Failure Rate	What percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?	ML Model Change Failure Rate can be expressed in the difference of the currently deployed ML model performance metrics to the previous model's metrics, such as Precision, Recall, F-1, accuracy, AUC, ROC, false positives, etc. ML Model Change Failure Rate is also related to A/B testing.

From https://ml-ops.org/content/mlops-principles

More complex pipelines

Where a regular software project mostly deals with code, an ML system involves data and models too:



From https://ml-ops.org/content/end-to-end-ml-workflow

In the following sections we go through the four metrics one by one and discuss what they mean for an ML system.

Deployment Frequency

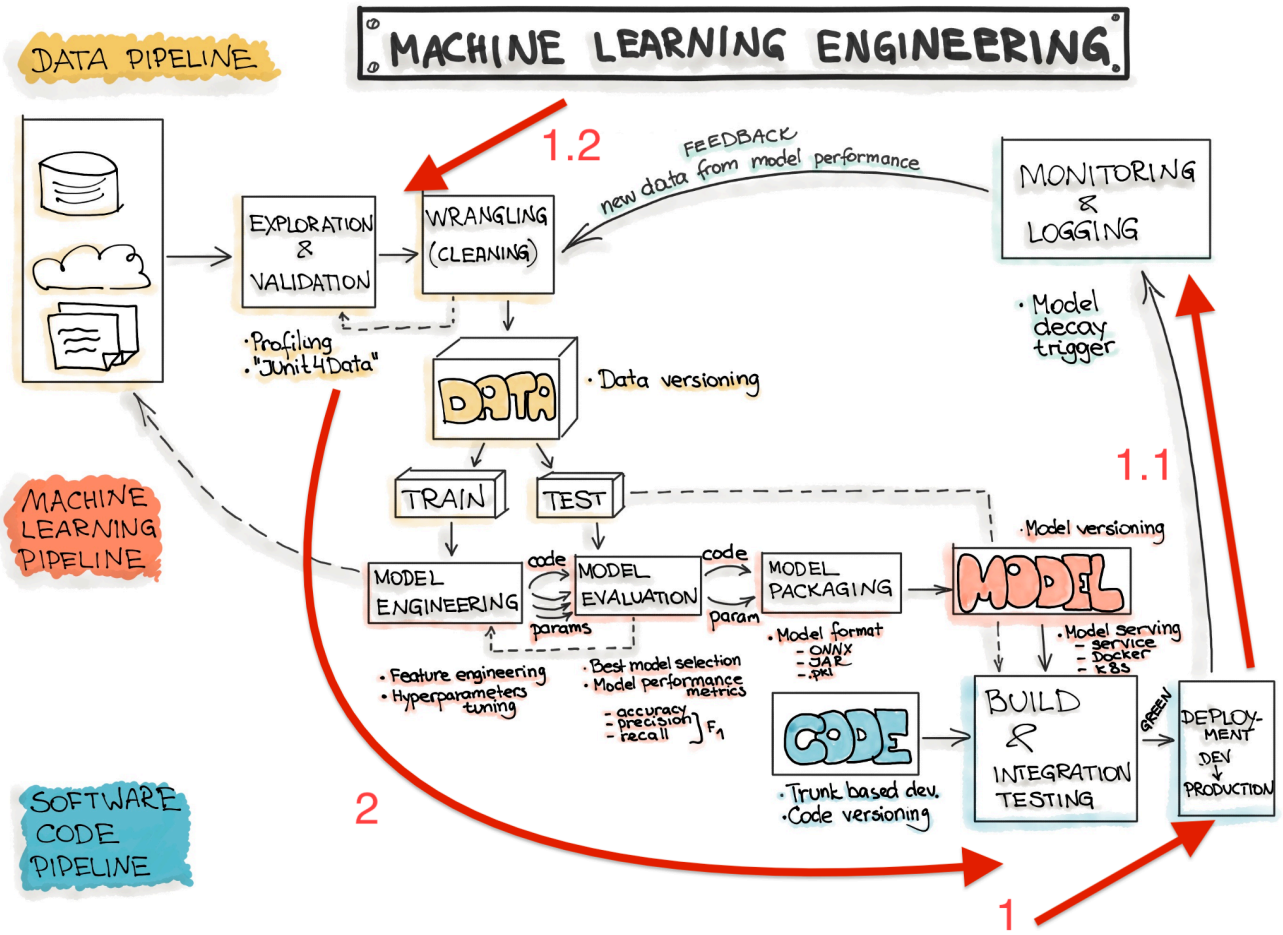
Deployment Frequency

How often do we deploy to end-users?

What deployment?

For a regular software project, a deployment ships code. But in an ML system, "a deployment" can mean several things:

- Data changes (new training data, new features)
- Model changes (new parameters, new architecture)
- Code changes (feature engineering, serving logic)



From https://ml-ops.org/content/end-to-end-ml-workflow

Which of these you count as "a deployment" affects how you measure the metric at all.

What affects frequency?

How often *can* you ship a new model? It depends on, among other things:

- The degree of automation in your pipeline
- How quickly you can validate that a new model is safe to ship
- How quickly the underlying data changes

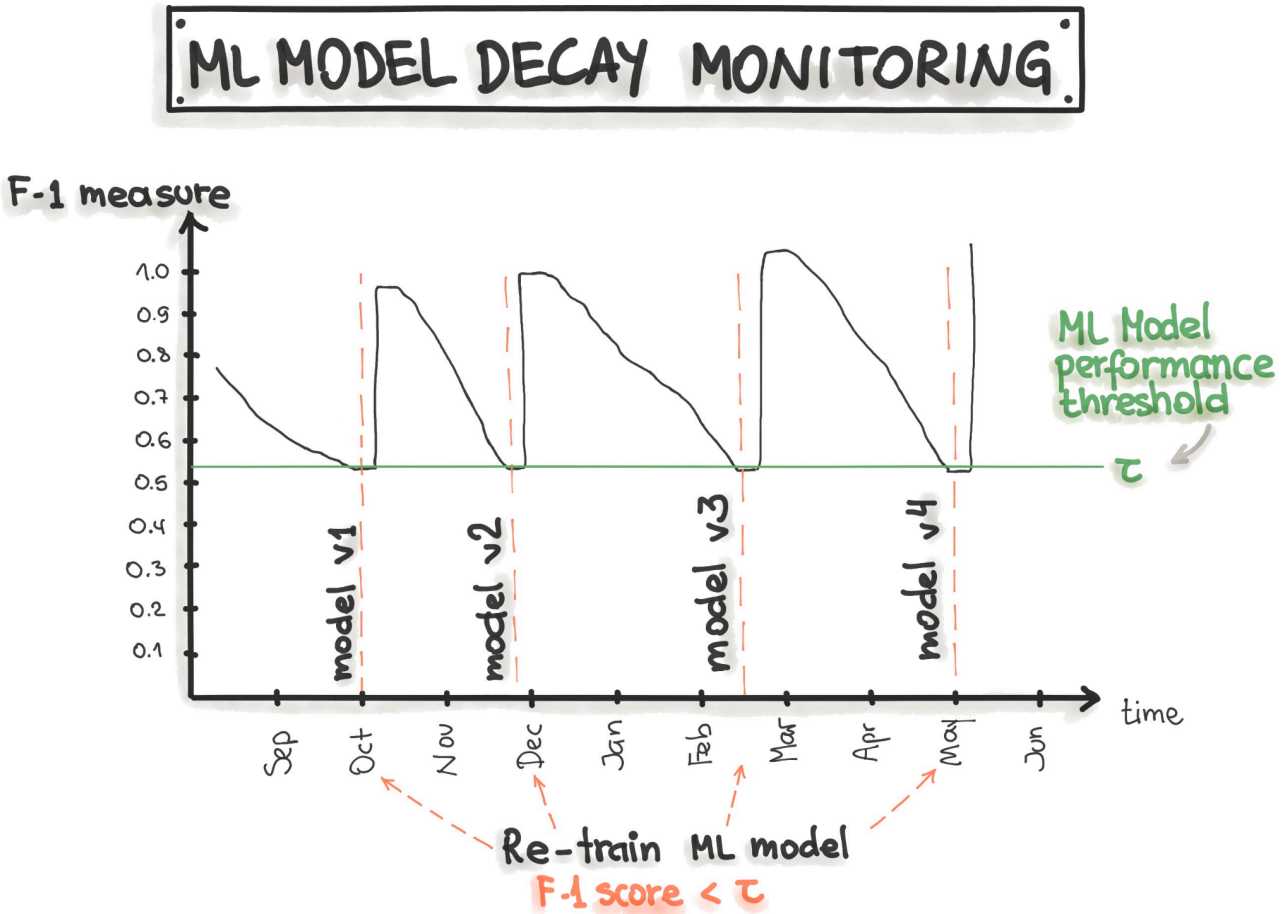
We covered the degree of automation in the last lecture – the more manual steps in the pipeline, the lower your effective deployment frequency.

Deployment Frequency: Model Issues

Model decay

A deployed model does not stay good forever – models decay over time, and the causes are many:

- Changes in dependencies
- New training data → worse model
- Different feature extraction
- Code updates
- ...

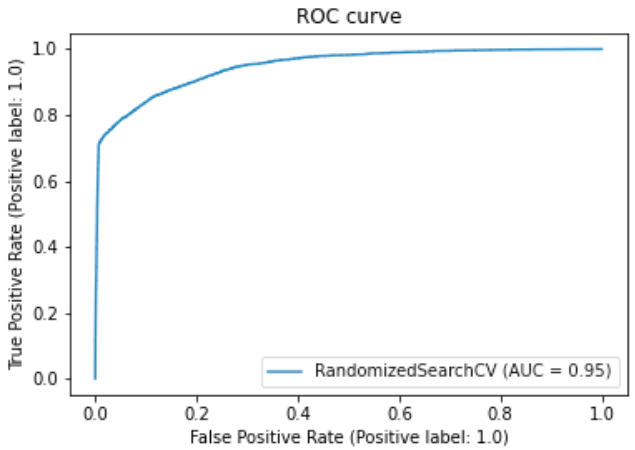


From https://ml-ops.org/content/mlops-principles

Quantifying changes

When the model or the world changes, how do we quantify it? Common model metrics to track:

- Training loss
- Accuracy / F1-score / ROC AUC
- Prediction bias
- Threshold ↔ business value (where you set the threshold determines the business trade-off)

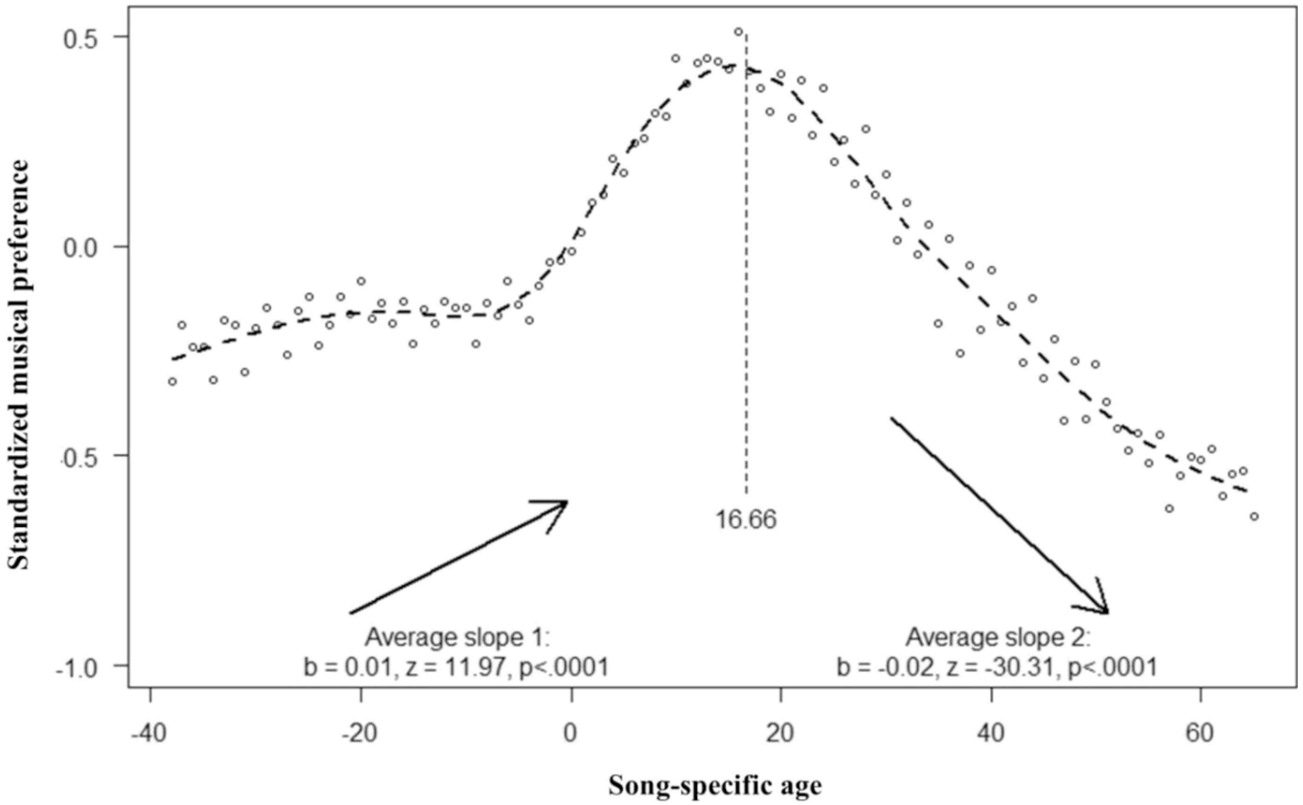


Concept/model drift

Drift is when the relationship between features and output changes – e.g., age ↔ music preference is not a stable relationship over time.

It can be:

- **Gradual** – slowly changing over time
- **Sudden** – a step change in the relationship
- **Recurring** – drifting back and forth, e.g. seasonally



Example: 30% week-over-week change. Fig. 3, from
https://link.springer.com/article/10.1007/s11002-022-09626-7

Deployment Frequency: Data Issues

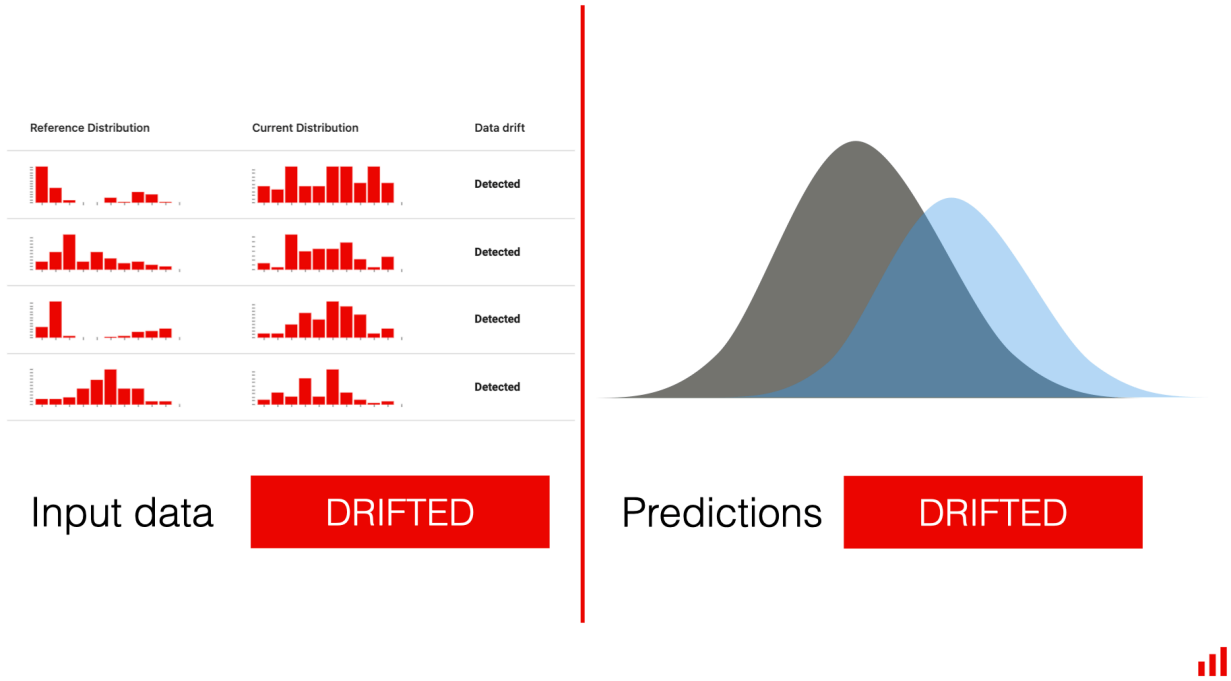
Deployment Frequency: Data Issues

It is not only the model that can cause trouble – the data flowing into it can too.

Data issues

Three main categories:

- **Data quality issues** – the data is wrong
- **Data/feature drift** – the data distribution changes over time
- **Outliers** – individual extreme values



From https://www.evidentlyai.com/ml-in-production/data-drift

Data quality

The list of possible data quality issues is long… for example:

- Data source changes
- Source data schema changes
- Data loss
- …

Common checks to catch them:

- Schema comparison
- NaNs, min/max, etc.

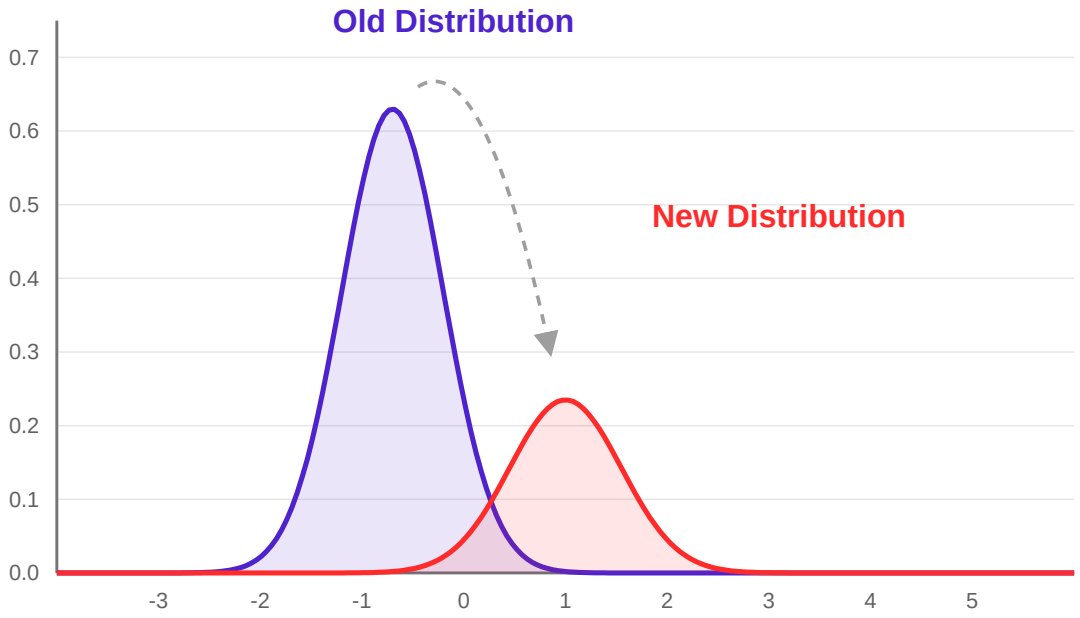
Data/feature drift

The distribution of your data changes over time.

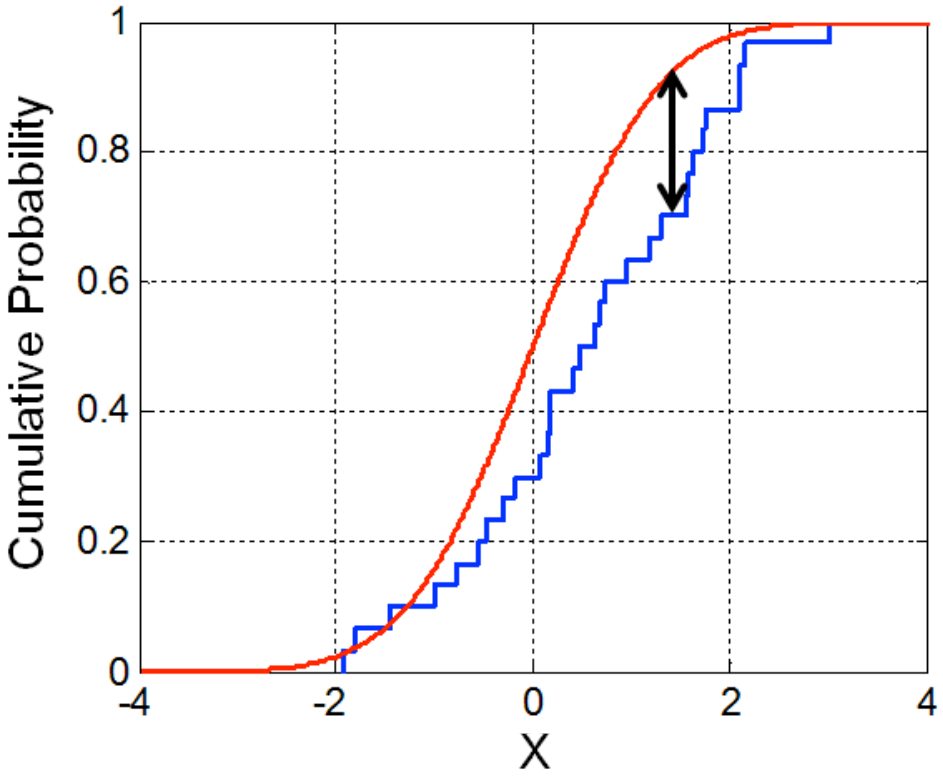
Question: do you compare against the source data distribution or the feature distribution?

Statistical tests to detect it:

- **Kolmogorov-Smirnov test** – i.e., the max distance between the cumulative distributions
- **Chi-squared** for categorical features



From https://datajello.com/production-ml-data-drift-and-concept-drift/



From
https://en.wikipedia.org/wiki/Kolmogorov%E2%80%93Smirnov_test#/media/File:KS_Example.png

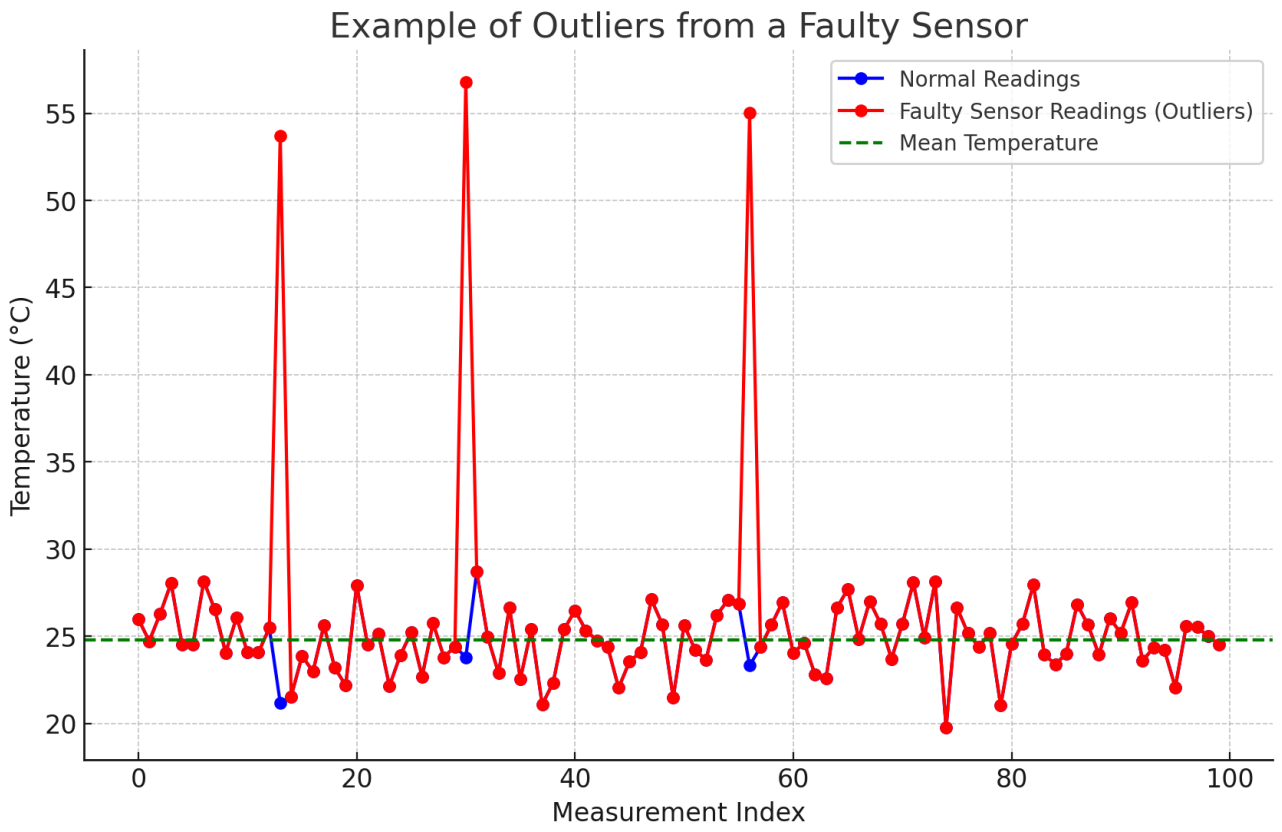
---

Outliers

Where do outliers come from?

- Bots / adversarial attacks?
- NaN / infinities?
- Faulty data entries?

→ The real question: **when and how to react?**



Deployment Frequency: Logging

Deployment Frequency: Logging

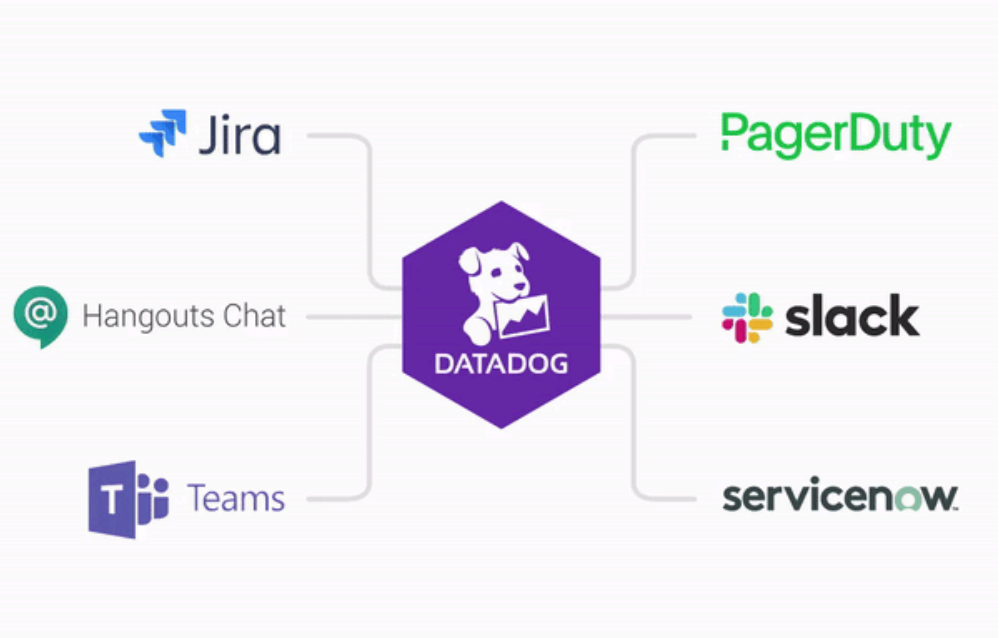
To even notice the issues from the previous pages – model decay, drift, outliers – you need data

about your system in production.

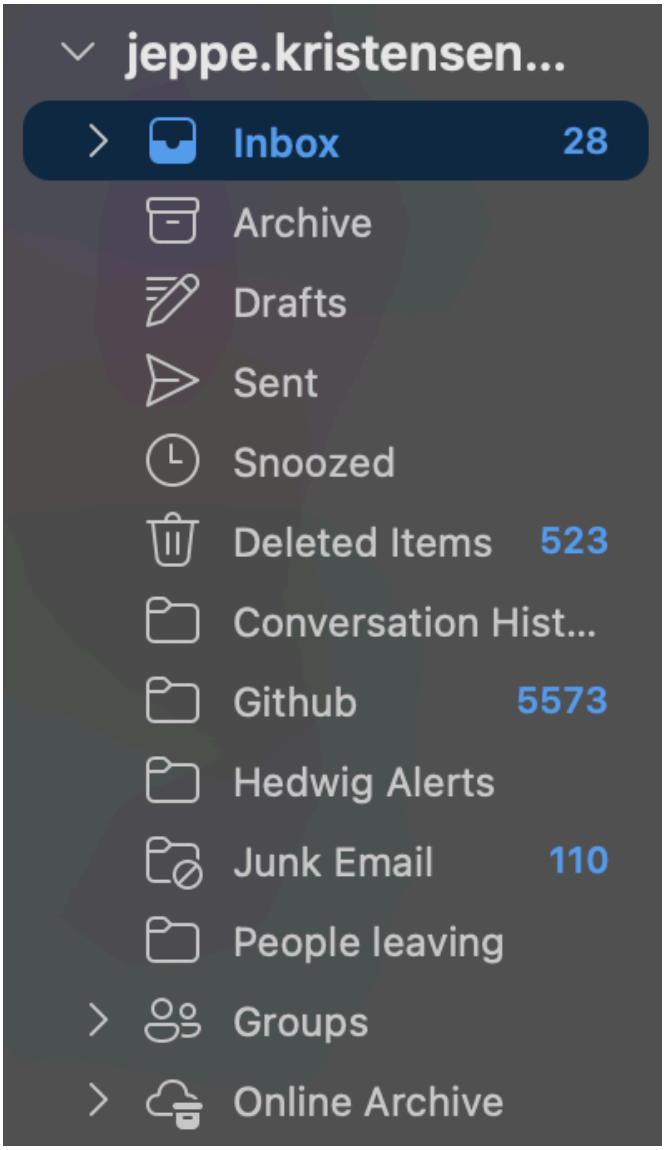
Log, monitor and alert for everything

Things worth logging, monitoring and setting up alerts for:

- Staleness of model
- Prediction quality
- Dev ↔ prod prediction mismatch
- Schema mismatches
- Numerical stability
- Computational performance
- Pipeline changes



Within reason, anyway.



This was my inbox 2 years ago. Now I don't even know how many Github notifications I have.

Circling back to deployment frequency

It is nice to know your average deployment frequency.

But it depends on lots of things – your product, your users, your data cadence, your level of automation.

→ Deployment frequency is a **good relative goal**: compare against yourself over time, not against other teams.

Lead Time for Changes

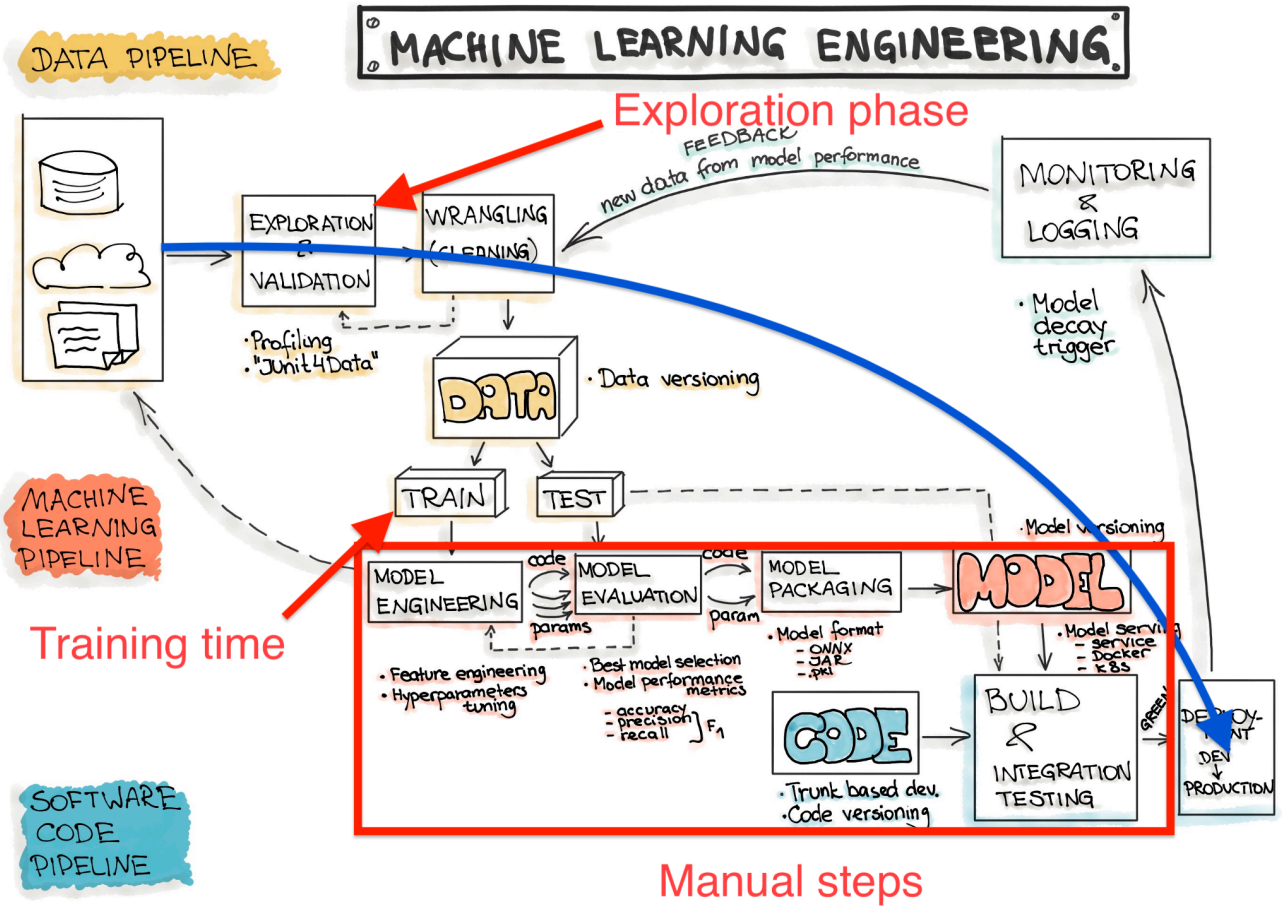
Lead Time for Changes

How long does it take from a change is committed until it runs in production?

What changes?

Just like with deployment frequency, "a change" in an ML system can be several things:

- Changes to the data or features
- Changes to the model
- Changes to the code



From https://ml-ops.org/content/end-to-end-ml-workflow

What affects the speed?

What makes the lead time from idea to production long or short?

- **Manual steps** – every hand-off adds waiting time
- **Exploration phase** – finding the right model takes experiments, not just engineering
- **Training time** – the model itself can take a long time to (re)train

The exploration phase is the ML-specific one – and it is what experiment tracking tools are designed to speed up. That is the topic of the next pages.

Lead Time for Changes: Experimentation

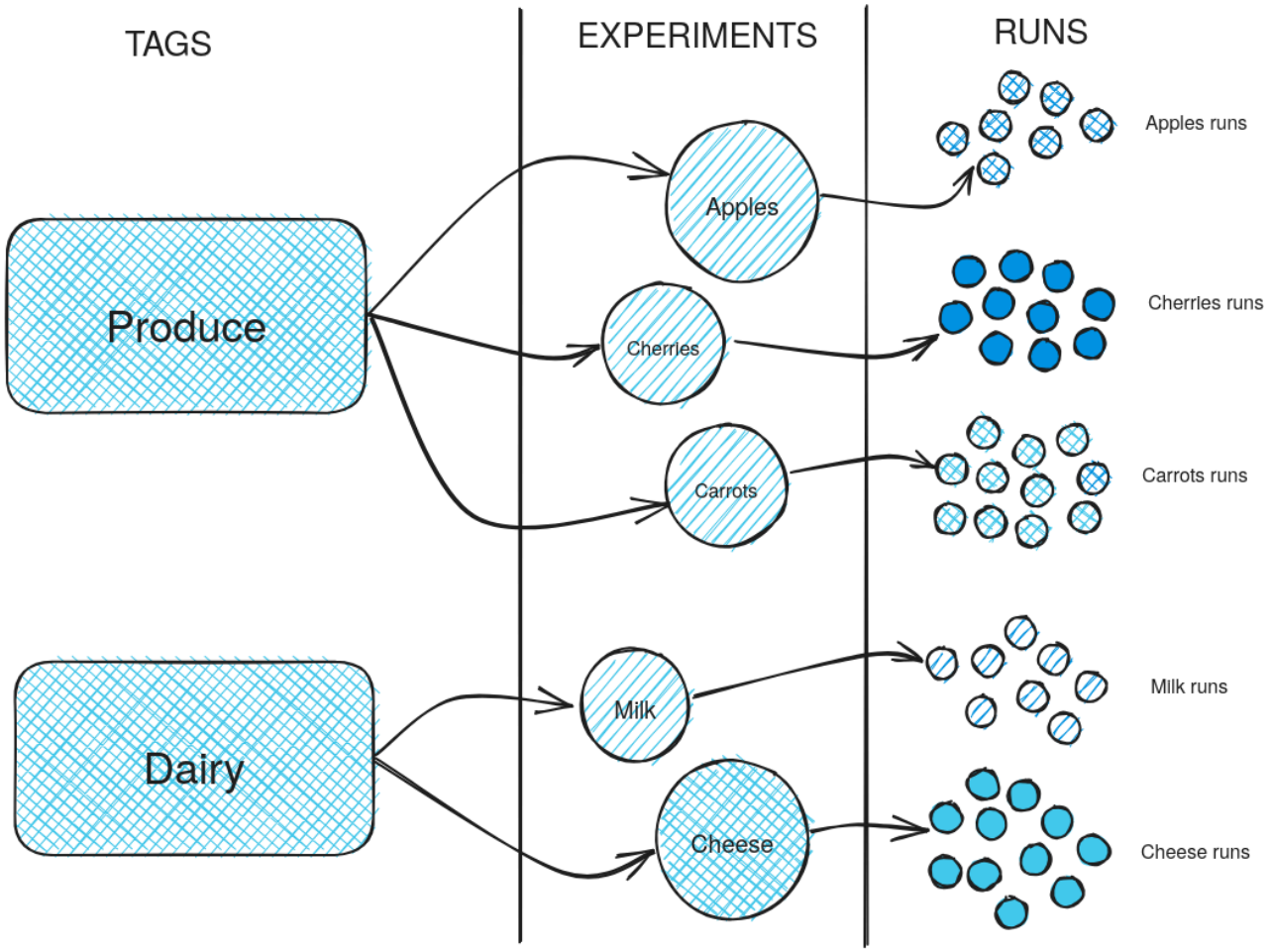
Lead Time for Changes: Experimentation

The exploration phase is where data scientists try out ideas: new features, other model types, different hyperparameters. Without structure, this becomes a folder of untracked notebooks called `final_v2_REALLY_FINAL.ipynb`.

Hierarchy of experiments

Tracking your experiments in a hierarchy keeps the exploration phase manageable:

- **Experiment** – the overall question you are investigating
- **Run** – one training attempt with specific parameters
- **Artifacts** – the model, metrics, plots, and configuration logged for each run



From https://mlflow.org/docs/latest/ml/getting-started/logging-first-model/step3-create-experiment/

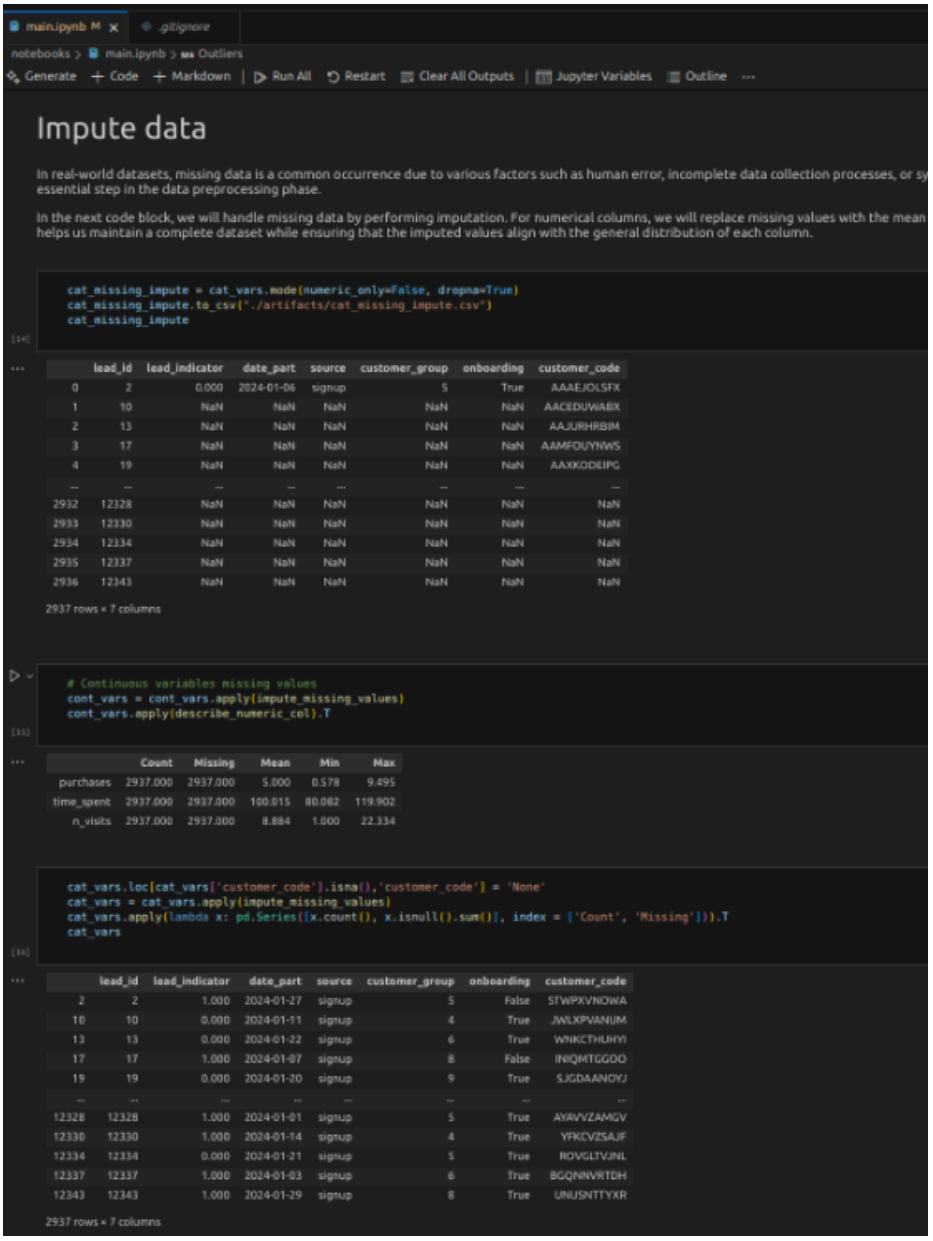


Exploration

What do we actually explore?

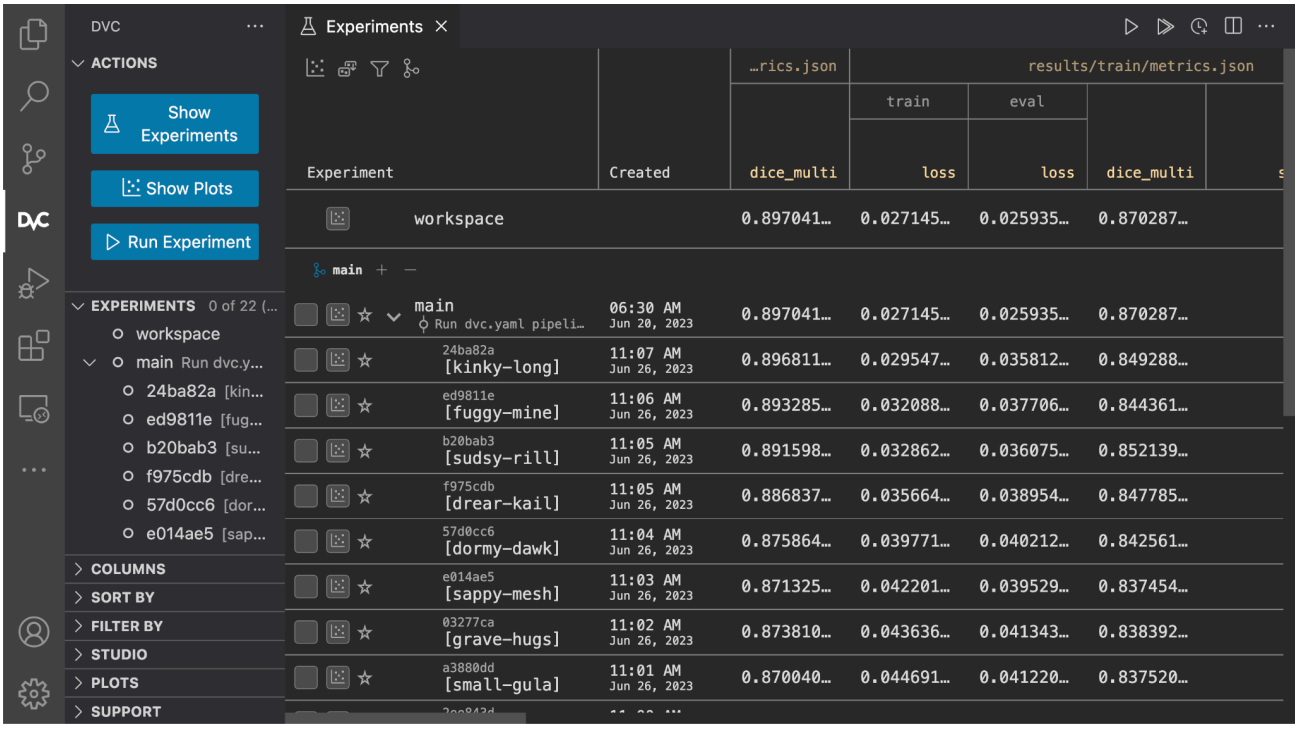
- New features/columns?
- Different output?
- Other kind of model?

→ Each of these becomes an **ML experiment**



Running an ML experiment – in DVC + VSCode

Experiment tracking is not unique to MLflow – DVC has its own experiment management, for example through the VSCode extension:



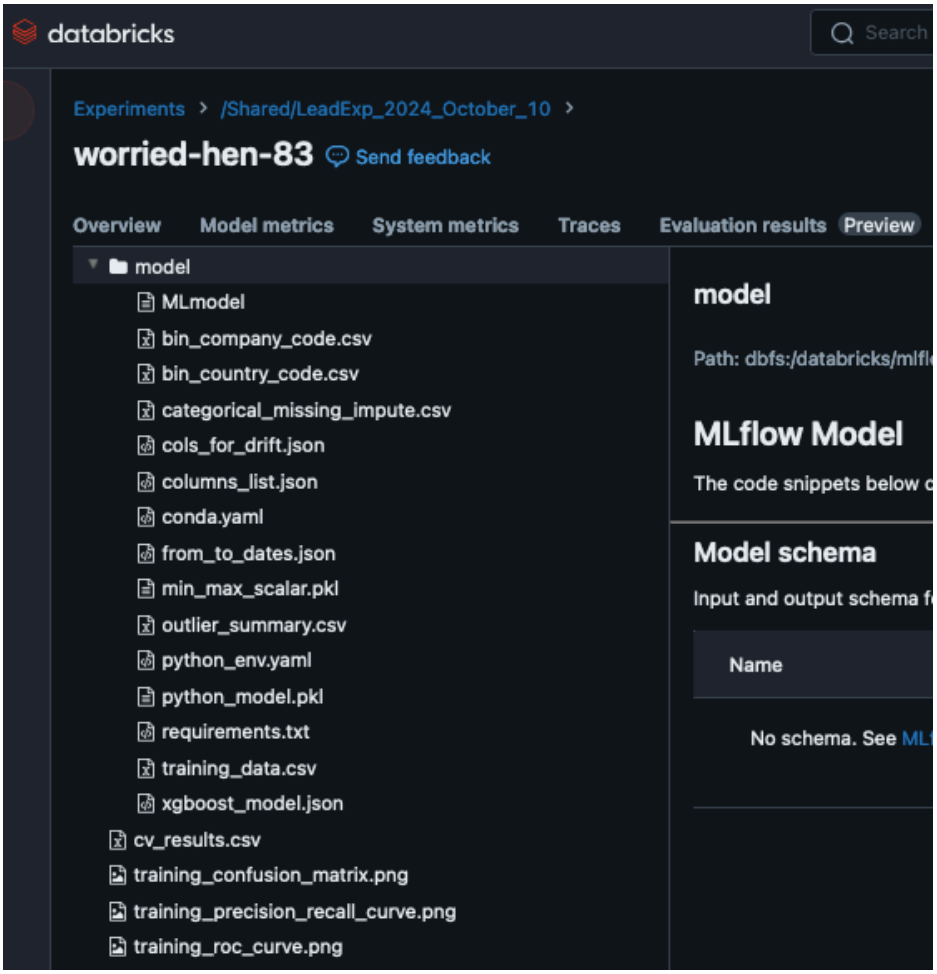
From https://dvc.org/doc/user-guide/experiment-management/comparing-experiments?tab=VSCode-Extension

Lead Time for Changes: In Production

Lead Time for Changes: Putting It in Production

Databricks ML experiments

At a larger scale, the same experiment-tracking concepts look like this – e.g. in Databricks:



Each experiment tracks:

- **Artifacts**
  - Model file
  - Training results
  - Configurations
- Managed MLflow implementation under the hood

Putting it in production

You have run many experiments – now what?

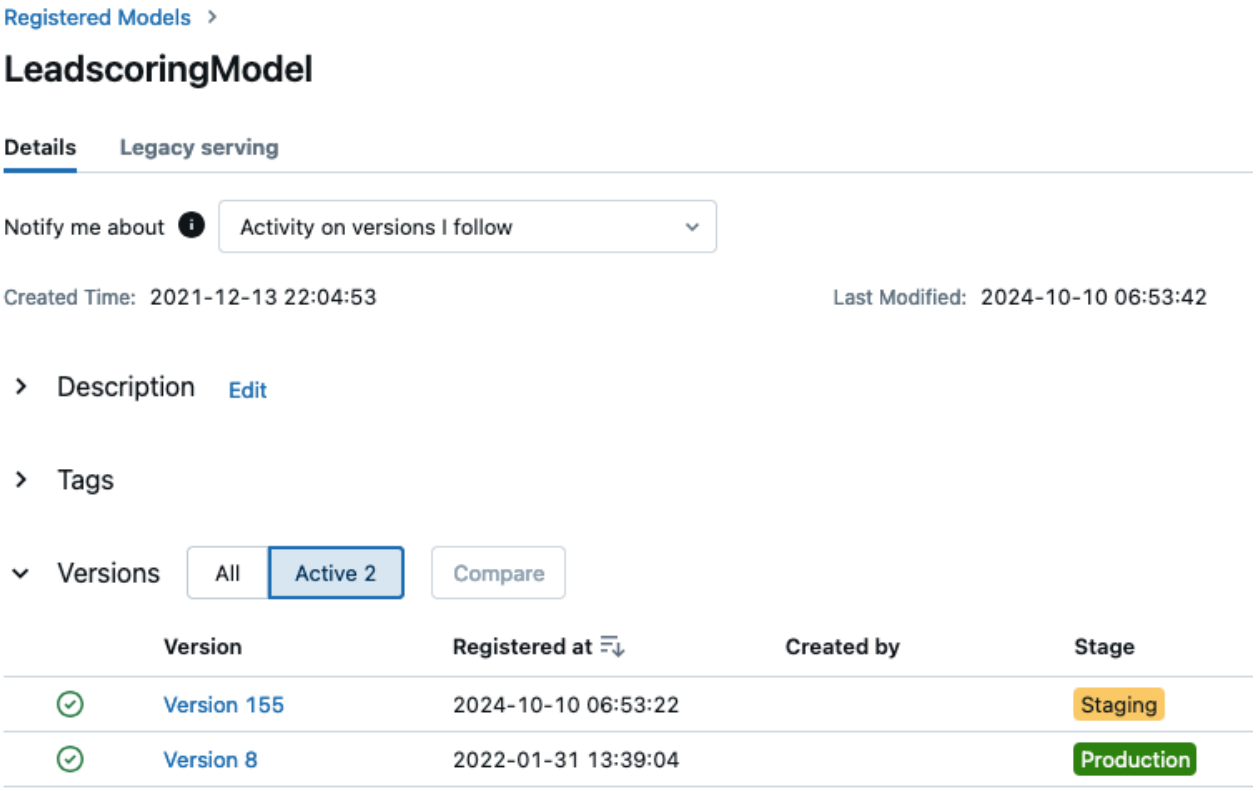
- **Multiple models** to choose between
- **Staging → production**: promote a model through stages before serving it to users
- **Go with the best metric one** – that is the obvious heuristic

But beware: **recent != better**.

The newest model was probably trained on the newest data, but:

- It might not be better on the metrics that matter
- The old model was not tested on the new data

Although, in this case it was...



Mean Time to Restore

Mean Time to Restore

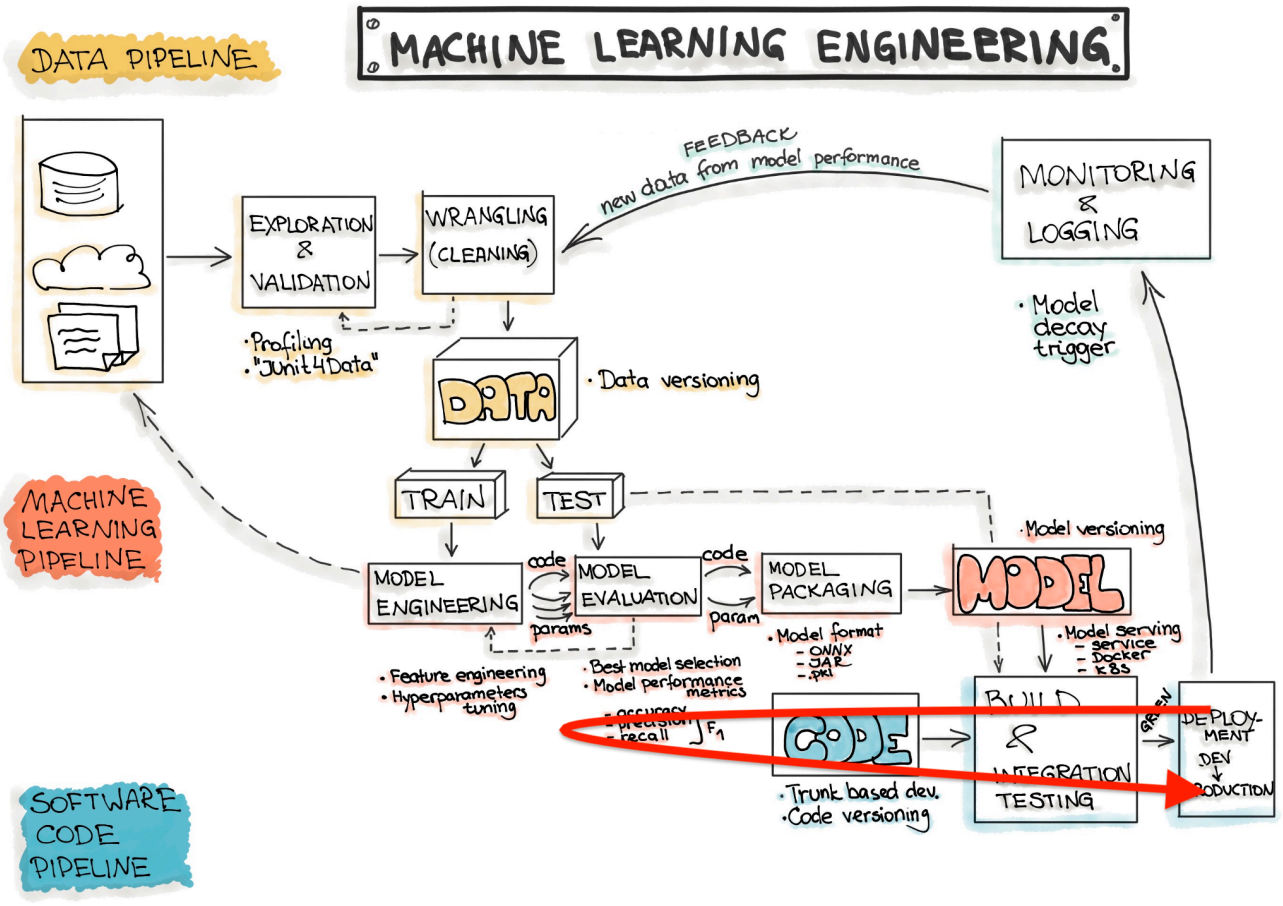
When something breaks in production – how long until it is fixed?

What does "restore" mean for ML?

For regular software, restoring means deploying a fix for a bug. For ML systems, an incident can be many things:

- A broken pipeline
- A model that suddenly performs poorly

- Bad or missing data reaching the model
- A serving issue



From https://ml-ops.org/content/end-to-end-ml-workflow

The end-to-end workflow gives you many places to look when something is wrong – and, importantly, more ways to recover: fix the code, fix the data, or swap the model.

Mean Time to Restore: Causes

Mean Time to Restore: Causes

New model releases

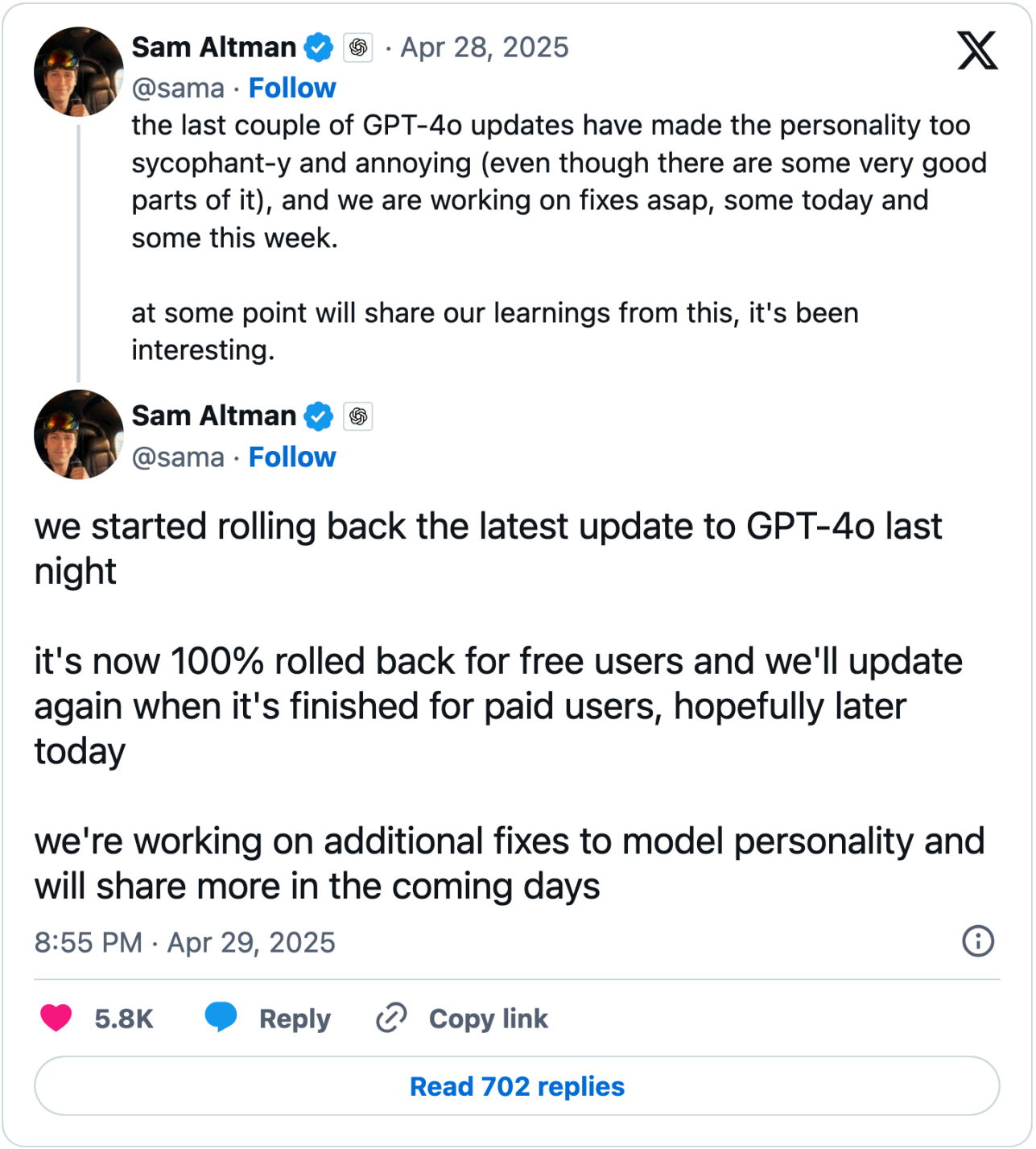
A nasty failure mode: you release a new model with **lower loss** on your offline metrics, but your

business KPI goes down too.

Why does that happen?

- Users build heuristics – they adapt to the model's behaviour, and a sudden change breaks their workflows
- Buggy implementation – the model is fine, the code around it is not
- Automation is not a guarantee – an automated pipeline happily ships a bad model

So: **retrain, rectify, or rollback?**



Troubleshooting

When something is off in production, where do you look?

- Bias in the model?
- Subgroup performance shift?
- Serving latency?

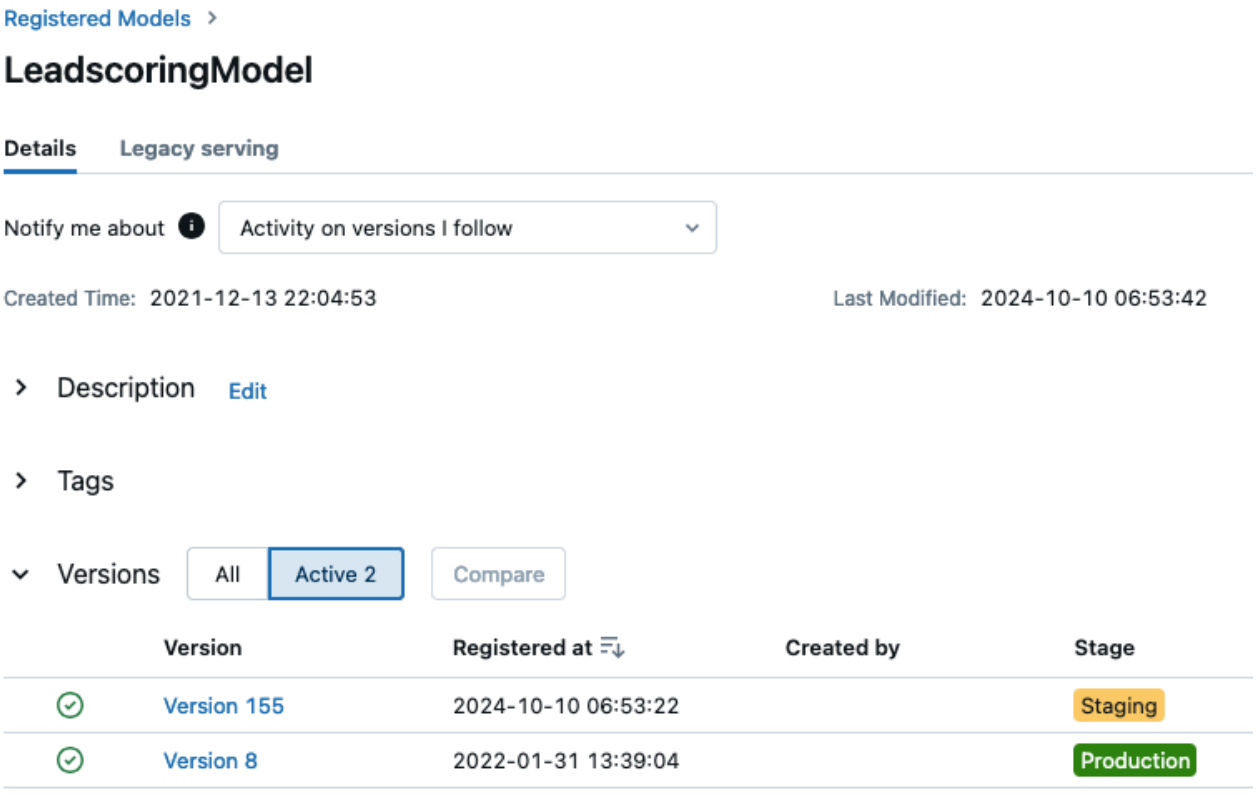
→ **Domain knowledge + logging** are what let you find the cause. Without good logging there is nothing to troubleshoot from, and without domain knowledge you don't know where to look.

Rollback of model

The fastest restore for a bad model release is often the simplest one:

- Promote the old model back to production
- This requires saving all model artifacts – every version of the model, its data and its configuration, must be retrievable

Note: rolling back gets you back to a working state, but it is **not solving the underlying problem**. You still need to figure out what went wrong before the next release.

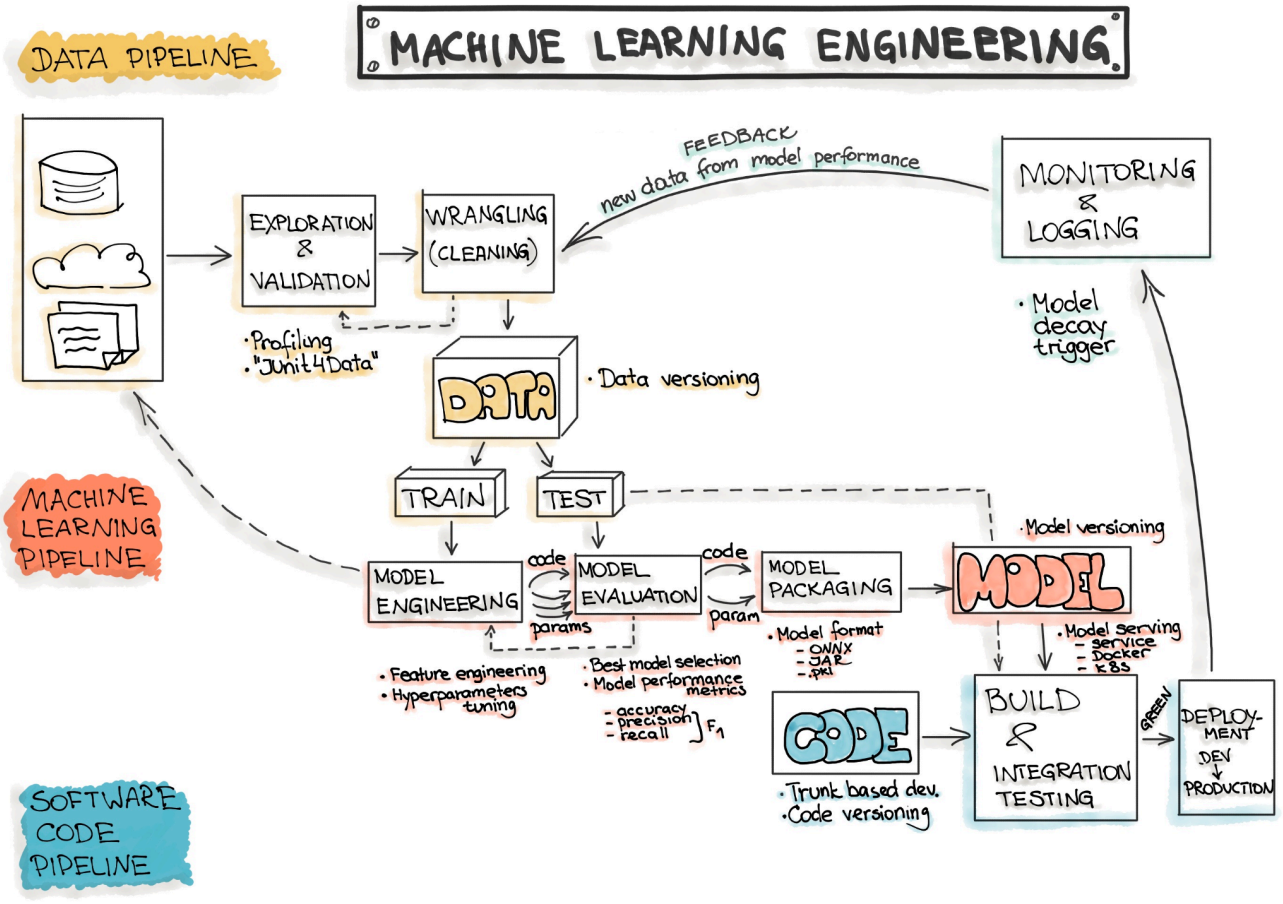


Change Failure Rate

How often does a change to production fail?

What is a failure for ML?

For regular software, a failure is usually clear: the service is down or misbehaves. For ML systems, a failure is more subtle:



From https://ml-ops.org/content/end-to-end-ml-workflow

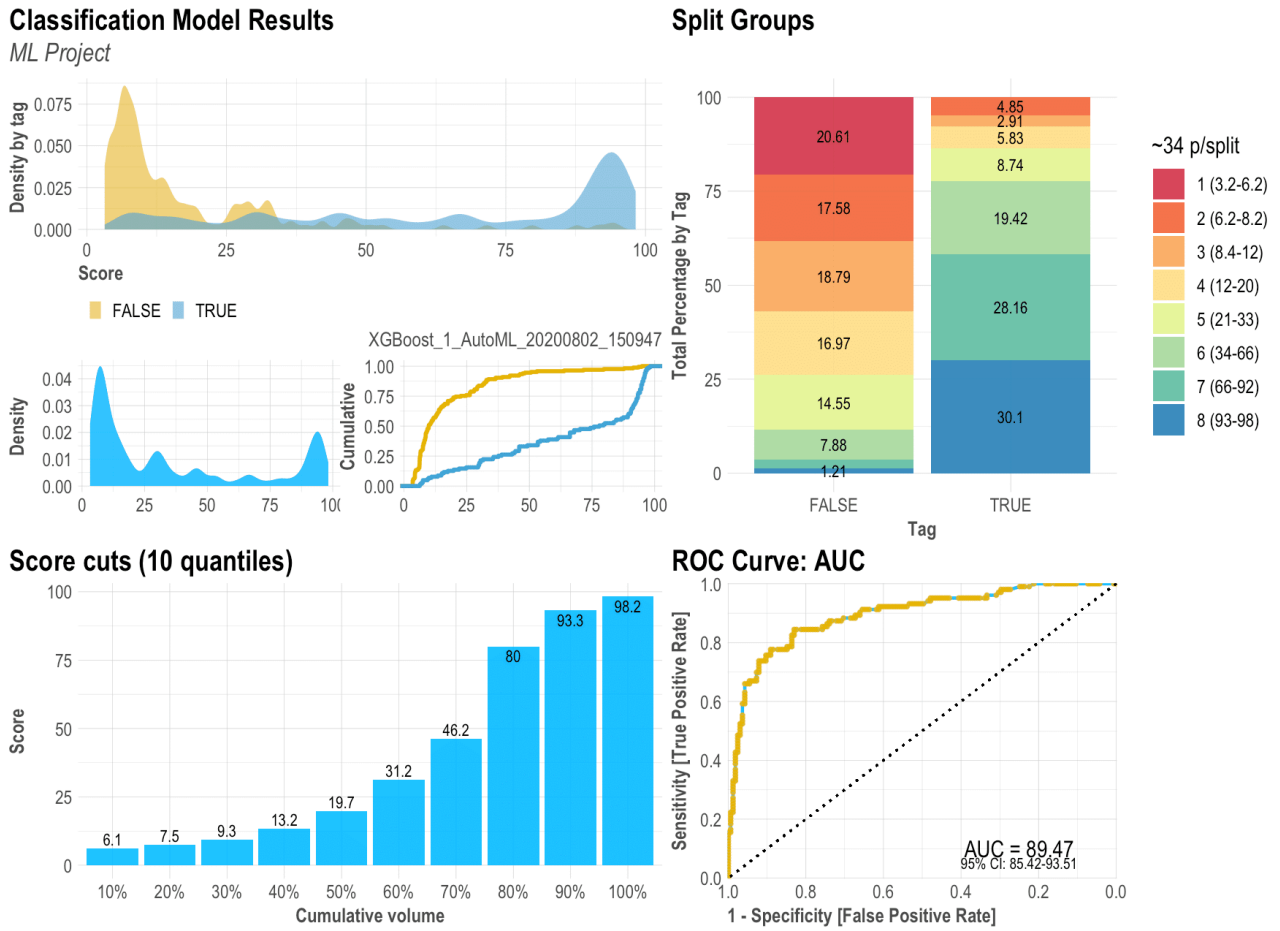
A change can "work" (no crash, predictions are served) and still be a failure if the model behind it performs worse than what it replaced.

Change Failure Rate: Managing Failures

Change Failure Rate: Managing Failures

Failures happen

- Any change carries a risk
- Failure = subpar performance:
  - F1-score, precision, etc
  - User experience
- Every release? Every 3rd? … how often do you accept to fail?

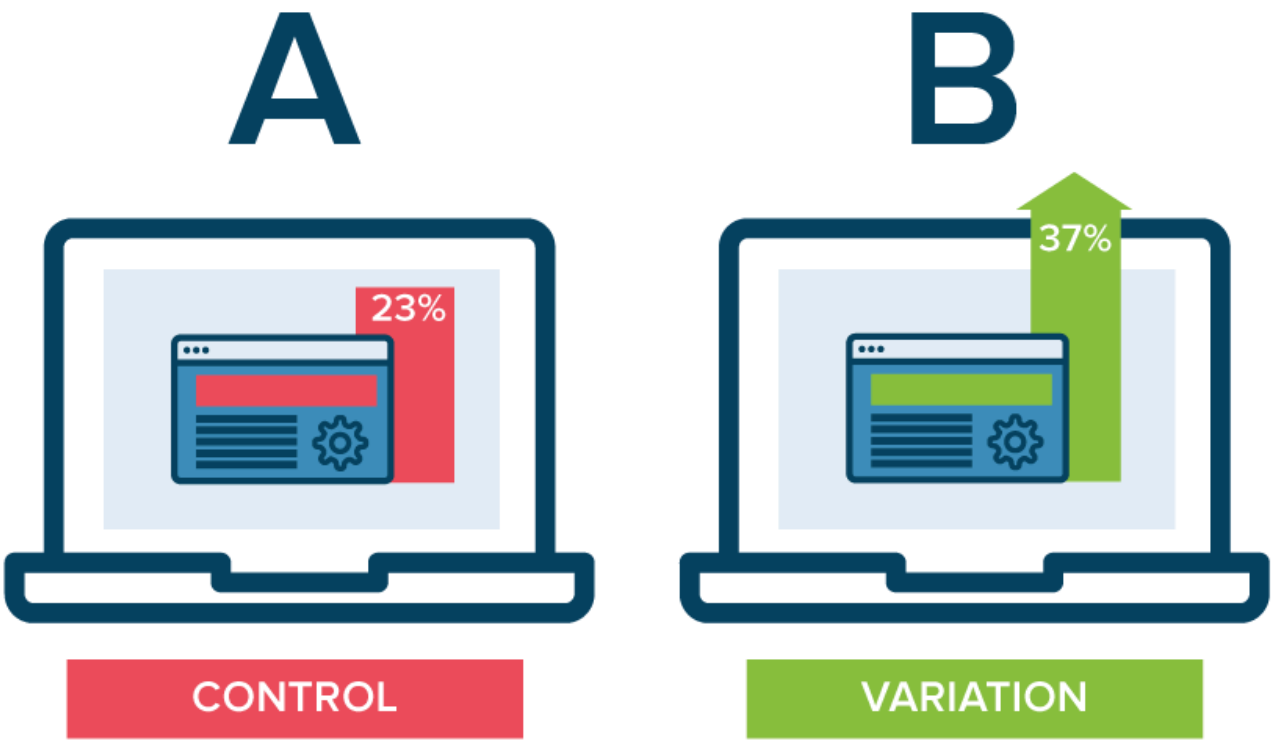


As discussed before – nothing is perfect, and models in particular can degrade quietly.

Managing risk

You don't have to roll a change out to everyone at once. Only rollout partly, with different strategies:

- **Pilot studies** – ship to a small, selected group first
- **A/B tests** – compare the new model against the old on random user groups
- **Shadow testing** – run the new model in production without serving its predictions, and compare offline



From https://mlinproduction.com/ab-test-ml-models-deployment-series-08/

As discussed before – gradual rollouts turn a potentially big failure into a small, contained one, and they give you the data to decide whether to continue.

Summary

Summary

We used the **DORA metrics for MLOps** as the red thread for why productionising models matters:

Deployment frequency

For ML, a deployment involves data, model and code – and the frequency of each is affected by automation, validation speed and data cadence. Model decay, data quality issues, drift and outliers all push you to deploy new models; logging and monitoring tell you when.

Lead time for changes

The exploration phase is the ML-specific bottleneck. Experiment tracking (MLflow, DVC, ...) turns scattered notebooks into comparable experiments with saved artifacts, and staging lets you

promote the best model – remembering that recent is not the same as better.

Mean time to restore

Failures in ML are subtle: a new model can have lower loss but worse business KPIs. Domain knowledge plus logging is how you troubleshoot, and saving all model artifacts is what makes a rollback possible.

Change failure rate

Any change carries a risk. Pilot studies, A/B tests and shadow testing let you roll out partly and contain the blast radius of a bad release.

MLOps?

Metric	DevOps	MLOps
Deployment Frequency	How often does your organization deploy code to production or release it to end-users?	ML Model Deployment Frequency depends on 1) Model retraining requirements (ranging from less frequent to online training). Two aspects are crucial for model retraining 1.1) Model decay metric. 1.2) New data availability. 2) The level of automation of the deployment process, which might range between *manual deployment* and *fully automated CI/CD pipeline*.
Lead Time for Changes	How long does it take to go from code committed to code successfully running in production?	ML Model Lead Time for Changes depends on 1) Duration of the explorative phase in Data Science in order to finalize the ML model for deployment/serving. 2) Duration of the ML model training. 3) The number and duration of manual steps during the deployment process.
Mean Time To Restore (MTTR)	How long does it generally take to restore service when a service incident or a defect that impacts users occurs (e.g., unplanned outage or service impairment)?	ML Model MTTR depends on the number and duration of manually performed model debugging, and model deployment steps. In case, when the ML model should be retrained, then MTTR also depends on the duration of the ML model training. Alternatively, MTTR refers to the duration of the rollback of the ML model to the previous version.
Change Failure Rate	What percentage of changes to production or released to users result in degraded service (e.g., lead to service impairment or service outage) and subsequently require remediation (e.g., require a hotfix, rollback, fix forward, patch)?	ML Model Change Failure Rate can be expressed in the difference of the currently deployed ML model performance metrics to the previous model's metrics, such as Precision, Recall, F-1, accuracy, AUC, ROC, false positives, etc. ML Model Change Failure Rate is also related to A/B testing.

From https://ml-ops.org/content/mlops-principles

Productionising models is not separate from the rest of MLOps – experimentation, selection and monitoring are the practices that keep the four metrics healthy in an ML system.

Exercise Intro

Background

The lecture used the DORA metrics as the red thread for why productionising models matters. Now we make it concrete: you will take a small model through the whole journey - track it, wrap it, serve it, and monitor it.

An ML experiment produces more than a model: it produces parameters, metrics, artifacts and (crucially) the *decision* about which configuration was best. Without a tool to record all of that, experiments live in ad-hoc scripts and spreadsheets: results are not comparable, not reproducible, and the model that ends up in production is "the one Jeppe ran last Tuesday". The tool we use for that is MLflow, which covers experiment tracking as well as a model registry and model serving; the lecture also mentioned alternatives such as Weights & Biases, Neptune and DVC.

Throughout the course you are building a real project (the itu-sdse-project repository) whose notebooks train a CNN on images of glass vials, register it in MLflow, and serve it behind an API. These exercises deliberately use a small tabular logistic regression instead - so you can see clearly what happens at each step - but the flow is the same one you will meet in the project: train, tune the classification threshold, wrap the model so the threshold travels with it (via MLflow's PythonModel API), register it, serve it, and watch for drift.

Learning Goals

> ***Learning goals*** *By the end of the exercises, we expect you to be able to do the following:*
>
> - *Organise and compare ML experiments using MLflow (tracking and the model registry)*
> - *Wrap a model with custom inference logic (such as a classification threshold) and log it as a proper model artifact*
> - *Serve a registered model, both with MLflow's scoring server and behind your own FastAPI API*
> - *Explain how to detect data drift and concept drift in a deployed model*

Exercises

The following exercises will guide you through productionising a model:

- Exercise 1: MLflow Catch-Up - Track ML experiments, register models, and select the best run

- Exercise 2: A Custom Model Wrapper - Wrap a logistic regression with a classification threshold
- Exercise 3: Serving the Model - Serve the registered model directly with MLflow, then wrap it in your own FastAPI app
- Exercise 4: Check Performance of Deployed Model - Monitor for data and concept drift

There are 6 python files in the **resources** folder for these exercises:

- example_model.py, the basic scikit-learn model the exercises build on: a logistic regression on the breast cancer dataset
- data_util.py, which provides the data: the breast cancer train/validation/test split and drift variants of it for Exercise 4 (imagine it is the output of the data processing pipelines)
- basic_mlflow_logging.py, a possible solution for Exercise 1, in case you get stuck
- basic_mlflow_training.py, a possible solution for Exercise 2, in case you get stuck
- basic_fastapi_app.py, a possible solution for Exercise 3, in case you get stuck
- basic_mlflow_evaluation.py, a possible solution for Exercise 4, in case you get stuck

If you get stuck, copy freely from the possible solutions - but as always, try yourself (and the MLflow documentation) first.

As always for the most effective learning, try to use the MLflow documentation and figure out how to solve the exercises yourself first before consulting AI.

Exercise 0: Setup

Make sure MLflow and friends are installed: `pip install mlflow` `pip install scipy` `pip install scikit-learn`

Try running it with: `mlflow --version`

Does this command run and show a version number of 2.18 or newer? Great! (FastAPI and uvicorn arrive with Exercise 3, so don't worry about them yet.)

In case you want run the MLflow for the project, you might have success with running a command like this:

```
--backend-store-uri sqlite:////Users/jeppekristensen/mlflow/mlflow.db \
--default-artifact-root /Users/jeppekristensen/mlflow/artifacts \
--host 127.0.0.1 --port 5000```
```

MLflow Basics

Exercise 1: MLflow Catch-Up

MLflow should be familiar to you at this point, so this first exercise is a quick catch-up to make sure everyone has a working setup and the core concepts (experiments, runs, parameters, metrics, artifacts, the model registry) are fresh. If you have never used MLflow before, don't panic - the exercise is a self-contained introduction, and everything you need is under the MLflow tracking quickstart.

MLflow has a lot of built-in "flavours" that make it easy to track models from well-known ML libraries such as scikit-learn or Tensorflow. That is what we use here - the *custom* models come in the next exercise.

Make sure MLflow and friends are installed (see Exercise 0 on the exercise intro page): `pip install mlflow` `pip install scipy` `pip install scikit-learn`

Exercise 1.1: Run an ML experiment

The resources/example_model.py script trains a `LogisticRegression` on the breast cancer dataset that ships with scikit-learn (`load_breast_cancer`) - a real binary classification task, and the same model you will keep building on in the following exercises. The script splits the data into a stratified 70/15/15 train/validation/test split - the same proportions the course project's training notebook uses. The three sets have distinct roles, and you will rely on the separation in the next exercises:

- **train**: fit the model
- **validation**: make decisions during experimentation (like picking a classification threshold)
- **test**: a final, untouched evaluation

1. **Edit the resources/example_model.py script to log the model run with MLflow**

   First `import mlflow`

   Then set the MLflow experiment via `mlflow.set_experiment("My experiment name")`

   Then start an MLflow run and move the training inside it, so the whole experiment - fit, predict, metric - happens within the run:

```
with mlflow.start_run() as run:
    lr = LogisticRegression(**params)
    lr.fit(X_train, y_train)

    y_pred = lr.predict(X_test)
    f1 = f1_score(y_test, y_pred)

    mlflow.log_params(params)
    mlflow.log_metric("f1_score", f1)
    mlflow.sklearn.log_model(lr, name="logistic_regression_model")
```

Finally, register the model in the model registry, pointing at the artifact you just logged. This two-step flow (log to the run, then register from the run) is exactly how the course project's training notebook does it:

```
mlflow.register_model(
    f"runs:/{run.info.run_id}/logistic_regression_model",
    name="breast_cancer_classifier",
)
```

2. ## Run training with different parameters

   For example set C in params to 0.1.

3. ## Compare the runs and select the best model

   You can compare the two runs via the printed output or via the MLflow UI:

   In the terminal run `mlflow ui`

   You can also select the best run *programmatically*, which is what the project's deployment notebook does when it picks the model to promote (there it orders by the test F1-score):

   ```
   runs = mlflow.search_runs(
       experiment_names=["My experiment name"],
       order_by=["metrics.f1_score DESC"],
   )
   best_run = runs.iloc[0]
   ```

   Why is selecting "the best run" a decision you should make in code rather than by eyeballing the UI?

4. ## What exactly is logged for each run?

   Check the mlruns directory or through the UI under "artifacts". For each run you should be able to find: the parameters you logged, the metric, and the model artifact together with its metadata (conda environment, the model's input signature if MLflow could infer one, ...).

By default everything is stored locally under `./mlruns`. MLflow can also log to a central place with `mlflow.set_tracking_uri(...)` - a database, or a tracking server your whole team shares. The course project, for instance, points MLflow at a local SQLite database so the run history lives in one file.

5. ### Try to use autologging

   Add `mlflow.autolog()` and remove other MLflow logging

   Look at what the autologged run contains compared to your explicit logging. Which approach gives you more control? Why might a production training job (like the project's) prefer explicit logging of exactly the parameters and metrics it cares about?

If all of that went smoothly: great, you are caught up. The interesting part starts in the next exercise, where a model is *more* than what a standard flavour can express.

Custom Models

Exercise 2: A Custom Model Wrapper

Sometimes you can't use a standard scikit-learn (or otherwise supported) model, so you have to define your own.

A very common case: a **classification threshold**. `LogisticRegression.predict()` classifies by picking the class with the highest predicted probability - for two classes, that means class 1 whenever `P(y = 1)` comes out above 0.5 - but 0.5 is not sacred. If one class is much rarer than the other, or a false positive is much costlier than a false negative, the *useful* threshold might be 0.3, 0.7, or anywhere else.

The threshold is a decision you make *with data* during experimentation, and it belongs *with the model* when you promote it to production - not hardcoded in some serving script by whoever deploys it. So in this exercise you will wrap the logistic regression from your `example_model.py` of Exercise 1 in a custom MLflow model that also stores the threshold, using MLflow's `PythonModel`:

Exercise 2.1: Question the default threshold

You already have a trained classifier: `example_model.py` from Exercise 1, with its train/validation/test split. Now start editing that script instead of starting over.

1. ### Warm-up: look at the probabilities

   Look at `predict_proba` for a few validation samples. What exactly does the model output?

   #### Hints:

   - `predict_proba` returns one row per sample and one column per class, in the order of `lr.classes_`. For our two classes that is `P(y = 0)` (malignant) and `P(y = 1)` (benign). Each row sums to 1, and for binary classification the interesting part is column 1, i.e. `predict_proba(X)[:, 1]`.
   - `predict` does not compute anything new - it picks the class with the highest probability (an argmax over the columns). For two classes that is exactly "class 1 if `P(y = 1) > 0.5`, else class 0" - at exactly 0.5 the tie goes to class 0. So `predict` is `predict_proba` with a hardcoded 0.5 decision rule - that default is what we are about to question.

2. ### Sweep thresholds

   Using the *validation* split, compute precision, recall and F1 for thresholds from e.g. 0.1 to 0.9 in steps of 0.05, based on `predict_proba`. Then check how the chosen threshold

performs on the *test* set.

A possible implementation:

```
def sweep_thresholds(lr, X_val, y_val):
    proba = lr.predict_proba(X_val)[:, 1]
    threshold_sweep = {}
    for threshold in np.arange(0.1, 0.91, 0.05):
        preds = (proba >= threshold).astype(int)
        threshold_sweep[round(threshold, 2)] = {
            "f1": f1_score(y_val, preds),
            "precision": precision_score(y_val, preds, zero_division=0),
            "recall": recall_score(y_val, preds, zero_division=0),
        }
    best_threshold = max(threshold_sweep, key=lambda t: threshold_sweep[t]["f1"])
    return best_threshold, threshold_sweep
```

Why do we sweep on the validation split rather than the test split?

Hint:

The test set is our stand-in for "unseen production data" - once we optimize against it, it stops being unseen. The course project tunes its threshold on the validation set in exactly this way, and additionally uses macro F1 because its classes are imbalanced.

3. ### Log the sweep to MLflow

   Log the whole threshold sweep as an artifact and the chosen threshold as a parameter, together with the usual training run: parameters, metrics (validation and test), and the (un-wrapped) model.

   Building on the MLflow code from Exercise 1, that could look like:

```
mlflow.set_experiment("My experiment name")
with mlflow.start_run() as run:
    best_threshold, threshold_sweep = sweep_thresholds(lr, X_val, y_val)

    mlflow.log_params(params)
    mlflow.log_param("threshold", best_threshold)
    mlflow.log_dict(threshold_sweep, "threshold_sweep.json")

    mlflow.log_metric("validation_f1_score", threshold_sweep[best_threshold]["f1"])
    test_preds = (lr.predict_proba(X_test)[:, 1] >= best_threshold).astype(int)
    mlflow.log_metric("test_f1_score", f1_score(y_test, test_preds))

    mlflow.sklearn.log_model(lr, name="logistic_regression_model")
```

Exercise 2.2: Wrap it in a `PythonModel`

1. ### Define the wrapper class

   Subclass `mlflow.pyfunc.PythonModel`. The wrapper holds both the fitted model *and* the threshold, and returns the predictions together with the threshold that produced them - a small taste of how the course project's wrapper returns predictions plus extra outputs (there: confidences, and a log of every request):

```
class ThresholdedLogisticRegression(PythonModel):
    def __init__(self, model, threshold=0.5):
        self.model = model
        self.threshold = threshold

    def predict(self, context, model_input):
        # MLflow injects `context` into every predict call of a pyfunc
        # model: read-only metadata about where and how the model is
        # being invoked, with the fields
        #   request_id           - ID of this inference request (e.g. when
        #                          called through the scoring server)
        #   is_evaluate          - True when the call comes from mlflow.evaluate
        #   dependencies_schemas - schemas of the model's dependencies
        #   model_id             - ID of the logged model being served
        #   endpoint_name        - name of the serving endpoint, when the
        #                          model is served behind one
        # We need none of it here - the threshold travels on the model
        # object - but the signature has to accept it.
        if isinstance(model_input, pd.DataFrame):
            model_input = model_input.values
        proba = self.model.predict_proba(model_input)[:, 1]
        return {
            "predictions": (proba >= self.threshold).astype(int).tolist(),
            "threshold": self.threshold,
        }
```

2. ### Log and register the wrapper

   Log the wrapper as the model artifact of the run with the pyfunc flavour:

```
with mlflow.start_run() as run:
    ...
    mlflow.pyfunc.log_model(
        name="thresholded_logistic_model",
        python_model=ThresholdedLogisticRegression(lr, threshold=best_threshold),
    )

mlflow.register_model(
    f"runs:/{run.info.run_id}/thresholded_logistic_model",
    name="thresholded_logistic_model",
)
```

Note that `python_model` is an *instance* - everything you store on it is serialized with the model. This is the same log-then-register flow you used in Exercise 1 and that the course project uses.

3. ### Load it back and verify

   Load the model from the registry (`mlflow.pyfunc.load_model("models:/thresholded_logistic_model/1")`) and predict on the test set.

   Hardcoding the version number is fine for this exercise. You can avoid the hardcoding by referring to registry stages and aliases instead (e.g. `models:/thresholded_logistic_model@production`).

   Verify that the loaded model's `predictions` equal what you get by manually thresholding `predict_proba` at your chosen threshold. What happens if you load the scikit-learn flavour instead - does it know anything about your threshold?

4. ### Experiment with the threshold

   Log a couple of runs with different thresholds (and the same underlying model). Look at the runs in the MLflow UI.

   *You just ran several "experiments" that share the same trained model but represent **different products**. Why does it make sense to track the threshold as a first-class parameter of a run?*

Model Serving

Exercise 3: Serving the Model

Your model now sits in the MLflow registry, threshold and all - but it isn't serving anyone yet.

The last step from "model" to "product" is putting it behind an API, so other systems (or humans) can get predictions on demand.

There are two ways to do that, and you will try both:

1. **Serve the model directly with MLflow.** `mlflow models serve` starts a scoring server for any logged model, `/invocations` endpoint included. This is exactly how the course project serves its vial-classification model.
2. **Define the API yourself with FastAPI.** When you need control over the contract - validation, documentation, response bodies - you can write the service yourself. A simple Python framework for that is FastAPI: it validates requests with pydantic, generates interactive documentation automatically, and is easy to test. The course project uses both together: a thin FastAPI app in front of the MLflow scoring server.

Exercise 3.1: Serve the model directly with MLflow

1. Start the scoring server

```
mlflow models serve -m models:/thresholded_logistic_model/1 -p 5001 --env-manager local
```

Under the hood this launches a FastAPI application run by uvicorn. You specify which model to serve with the `-m` flag, which accepts any model URI: a registry version like here, or a `runs:/...` URI.

The `--env-manager local` flag tells MLflow to serve the model in your current Python environment. Without it, MLflow first builds a fresh virtual environment from the model's logged requirements - safer, but slow, and often the first thing that breaks on a laptop.

2. Call the invocations endpoint

Your model expects rows of 30 features - real measurements of cell nuclei, not numbers you would want to type by hand. Two example rows (a malignant sample and a benign one) are ready in resources/payload.json, and resources/payload.csv shows the same two rows with their column names. Download the JSON and call the endpoint with it:

```
curl -X POST http://localhost:5001/invocations \
  -H "Content-Type: application/json" \
  -d @payload.json
```

The scoring server converts the JSON into a DataFrame and passes it to your wrapper's `predict`. The response is your wrapper's output, wrapped by the scoring server in a `predictions` key:

```
{"predictions": {"predictions": [0, 1], "threshold": 0.25}}
```

Where did the threshold in the response come from? What happens if you send the wrong field name, or a string instead of a number?

3. Check the health endpoint

```
curl -i http://localhost:5001/health
```

Is the response valid JSON? This bare-newline answer is exactly why the course project puts its own FastAPI app in front of the scoring server: to control what the endpoints actually respond. Keep that motivation in mind for the next part.

Exercise 3.2: Define your own API with FastAPI

1. Install and run a hello-world app

`pip install fastapi uvicorn`

Create `app.py`:

```
from fastapi import FastAPI

app = FastAPI(title="Model service")

@app.get("/health")
def health():
    return {"status": "ok"}
```

Run it with `uvicorn app:app --reload` and open http://localhost:8000/health - then http://localhost:8000/docs.

What is the /docs page? (hint: OpenAPI). Why does a health endpoint make sense to have? Think back on the logging/monitoring part of the lecture

2. The skeleton of the model service

It's not a learning objective to set up a fully functioning API service. However, we want to give enough context for what it takes to serve a model via an API. Here is therefore a skeleton of the full service. Replace your `app.py` with it and read through: it loads your registered model when the app starts, defines the request and response contract with pydantic, and answers health checks. The `invocations` endpoint is yours to finish in the next step.

```
import os
from contextlib import asynccontextmanager

import mlflow
import pandas as pd
from fastapi import FastAPI
from fastapi.responses import JSONResponse
from pydantic import BaseModel

MODEL_URI = os.environ.get("MODEL_URI", "models:/thresholded_logistic_model/1")


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the model once at startup, not per request
    app.state.model = mlflow.pyfunc.load_model(MODEL_URI)
    yield


app = FastAPI(title="Thresholded classifier service", lifespan=lifespan)


class Invocation(BaseModel):
    # One or more rows of features (30 features per row for this model)
    features: list[list[float]]


class Prediction(BaseModel):
    # The wrapper's output: predictions plus the threshold that produced them
    predictions: list[int]
    threshold: float


@app.get("/health")
def health():
    # "ok" only once the model is actually loaded and ready to serve
    if getattr(app.state, "model", None) is None:
        return JSONResponse(status_code=503, content={"status": "loading"})
    return {"status": "ok"}


@app.post("/invocations")
def invocations(payload: Invocation) -> Prediction:
    ...  # your job in the next step
```

3. ## Make the invocations endpoint work

   The endpoint body is only two lines: create a DataFrame from the request features, and feed it to the loaded model for the response.

```
@app.post("/invocations")
def invocations(payload: Invocation) -> Prediction:
    df = pd.DataFrame(<payload operation>)
    predictions = <app.state.model prediction operations>
    return Prediction(<predictions output format>)
```

`payload` is an `Invocation` item, so you need to get the `features` values out somehow.

Start the service with `uvicorn app:app --reload` and try it - reuse the payload from 3.1, only with the other field name (FastAPI expects `features` where the scoring server expected `instances`):

```
python -c "import json; d=json.load(open('payload.json')); json.dump({'features': d['instances

curl -X POST http://localhost:8000/invocations \
  -H "Content-Type: application/json" \
  -d @payload_fastapi.json
```

Compare the response to the one from the scoring server in 3.1: here it is not double-wrapped - defining the API yourself means controlling the contract. What happens if you send a string instead of a number? Or the wrong field name? Watch what FastAPI does for you - and discuss why "sanity check input/output" from the lecture matters.

Solution hints

- For `<payload operation>`: the request rows are in `payload.features` - pydantic has already parsed and validated the JSON for you.
- For `<app.state.model prediction operations>`: `app.state.model.predict(df)` returns a dict with `predictions` and `threshold`.
- For `<predictions output format>`: unpack that dict into the response model with `**`, so `Prediction(**predictions)`.

Model Monitoring

Exercise 4: Check Performance of Deployed Model

Once a model is deployed, you need to monitor its performance. Here we test how we can log two things: **data drift**, which is when your input variables drift, and **concept drift**, which is when the fundamental relationship between your dependent and independent variables changes.

We continue with the thresholded classifier from the previous exercises. `resources/data_util.py` provides two new datasets for this, built from the same breast cancer data: `get_breast_cancer_drift_data()`, where the measurement device has drifted so that every reading comes out 25% higher, and `get_breast_cancer_concept_data()`, where a new labeling guideline has flipped every diagnosis.

In these exercises the test set (from `get_breast_cancer_data()`) plays the role of "the data the model was approved on": it is the baseline we compare new data against, and it is where we evaluate performance. The course project does the same thing more seriously: its training notebook stores a set of baseline statistics, and the serving endpoint compares live traffic against that baseline.

Exercise 4.1: Evaluate on drifted data

1. Load your registered model

Load the wrapper with `mlflow.pyfunc.load_model("models:/thresholded_logistic_model/1")` - the threshold comes with it, as it should.

You logged the model in Exercise 2 - check in the MLflow UI what version your model has ended up on.

2. Evaluate the model on both new datasets

Both new datasets come with labels, so you can measure performance directly. Log one run per dataset:

```
datasets = {
    "data_drift": get_breast_cancer_drift_data,
    "concept_drift": get_breast_cancer_concept_data,
}

for name, loader in datasets.items():
    with mlflow.start_run(run_name=name):
        X_new, y_new = loader()
        output = model.predict(pd.DataFrame(X_new))
        y_pred = output["predictions"]
        mlflow.log_metric("accuracy", accuracy_score(y_new, y_pred))
        mlflow.log_metric("f1_score", f1_score(y_new, y_pred))
```

For a model that returns plain predictions (a standard flavour), `mlflow.evaluate` can compute and log all of this for you in one call. Why do we extract `predictions` and compute the metrics ourselves here? (Hint: look at what your wrapper returns.) Why is this useful to wrap this in MLflow?

3. ## Which test detects which drift?

   Here are two drift detectors. Look at what each of them compares, and figure out which one detects the data drift and which one can detect the concept drift. Then add the matching test call to each of your two runs and log the result as a metric.

```
def detect_foo_drift(X_old, X_new):
    """Kolmogorov-Smirnov test"""
    drifted_features = []
    for i in range(X_old.shape[1]):
        res = scipy.stats.kstest(X_old[:, i], X_new[:, i])
        if res.pvalue < 0.05:
            drifted_features.append(i)
    return drifted_features

def detect_bar_drift(y_old, y_new):
    """Chi-square test"""
    classes = np.unique(np.concatenate([y_old, y_new]))
    old_counts = np.array([(y_old == c).sum() for c in classes])
    new_counts = np.array([(y_new == c).sum() for c in classes])
    expected = new_counts.sum() * old_counts / old_counts.sum()
    res = scipy.stats.chisquare(f_obs=new_counts, f_exp=expected)
    return res.pvalue
```

   Which dataset does each test fire on? Why does the feature test stay quiet for the concept drift data even though the model's error explodes?

What you should see

On the data drift dataset: the feature test fires on about 26 of the 30 features, the label test stays quiet, and accuracy drops from about 0.97 to about 0.74. On the concept drift dataset: the feature test stays quiet, the label test fires (p-value near zero), and accuracy collapses to about 0.05 - the inputs are unchanged, but the outcomes flipped.

4. ## Discuss: measuring drift in production

   In these exercises the new data arrived as a ready-made file, labels included, and each drift check was a handful of lines of code. In production both drifts are harder to measure - not because the statistics get harder, but because the data to run them on has to be collected somewhere.

   - **Data drift** only involves inputs, so in principle your serving service sees everything it needs on every request. What would you have to log to keep the KS tests running on

live traffic?

- **Concept drift** only showed up through the labels: the feature distributions were fine, the class balance flipped. The test above needs true labels for new data - where do they come from?

What it takes in practice

For **data drift**:

- Log every request: the feature values, the prediction, a timestamp, and which model version served it - without the features there is nothing to test.
- Store the baseline (the test set, or summary statistics saved by the training job) so live data has something to be compared against.
- Aggregate requests into time windows and run the tests per window, alerting on drift counts or p-values - never on individual requests.

For **concept drift**:

- True labels arrive late (a confirmed diagnosis, a returned order, ...): join the delayed ground truth back to the logged predictions and track windowed accuracy/F1 over time.
- Watch the true label distribution itself: if inputs and predictions are stable but the outcome share moves, the relationship underneath has changed.
- Monitor calibration: predicted probabilities that no longer match observed frequencies are an early concept-drift symptom.
- Sample predictions for human review where labels never materialize - and expect the remedy to be relabeling or retraining, not a "fix" of the model.

And that is it! Hopefully these exercises gave you an idea how you can deploy and serve a model via MLflow/FastAPI. You might see more complex setups in production systems (including the project), but now all the exercises should cover the main parts you need for the project!

Back to Introduction